

“WOMEN SAFETY APP – SOS AND LIVE TRACKING”

A Project Report submitted in partial fulfilment of the requirements for the award of the
degree of

BACHELOR OF TECHNOLOGY

in

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

II Year B.Tech – I Semester

By

D MANYA AKSHITHA : 2520090032

P NIRVIGHNA SAI : 2520090144

R BHANU PRIYA : 2520090171

Under the Esteemed Guidance of

Dr. M Gayatri

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Academic Year 2026 – 2027

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

Declaration

The Project Report entitled “WOMEN SAFETY APP – SOS AND LIVE TRACKING” is a record of bonafide work of D MANYA AKSHITHA – 2520090032, P NIRVIGHNA SAI – 2520090144 and R BHANU PRIYA – 2520090171, submitted in partial fulfilment for the award of B.Tech in Computer Science and Information Technology to K L University. The results embodied in this report have not been copied from any other department, university or institute.

D MANYA AKSHITHA – 2520090032

P NIRVIGHNA SAI – 2520090144

R BHANU PRIYA – 2520090171

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the project-based report entitled “WOMEN SAFETY APP – SOS AND LIVE TRACKING” is a bonafide work done and submitted by D MANYA AKSHITHA – 2520090032, P NIRVIGHNA SAI – 2520090144 and R BHANU PRIYA – 2520090171, in partial fulfilment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in the Department of Computer Science and Information Technology, K L (Deemed to be University), during the academic year 2026 – 2027.

Signature of the Guide

Dr. M Gayatri

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance of many people. We wish to express our sincere thanks to all those who have assisted us in one way or another in completing the project.

Our greatest appreciation goes to our guide, Dr. M Gayatri, Department of Computer Science and Information Technology, for her tremendous support, encouragement and guidance throughout this project. Her suggestions at every review helped us to improve the application and this report.

We also express our gratitude to our Course Coordinator and to the Head of the Department of Computer Science and Information Technology for providing us with the facilities and the opportunity to complete this project-based work.

We thank all the teaching and non-teaching staff members who have helped us directly or indirectly in the successful completion of the project. Finally, we sincerely thank our parents, friends and classmates for their kind help, cooperation and encouragement during our work.

D MANYA AKSHITHA – 2520090032

P NIRVIGHNA SAI – 2520090144

R BHANU PRIYA – 2520090171

Abstract

Personal safety is a daily concern for women who travel alone, commute late at night or pass through isolated areas. In an emergency, traditional measures such as dialling a number and explaining the situation can be slow and unreliable. This project, “Women Safety App – SOS and Live Tracking”, is a mobile application that enhances personal safety through an instant SOS alert and a live location tracking mechanism.

By pressing a single SOS button, the user sends her real-time GPS coordinates and an emergency alert to pre-selected trusted contacts, without having to explain the situation while under threat. After the alert, the live tracking feature continuously shares the user’s movement with her guardians until the emergency is resolved, which improves transparency and helps families and responders to locate her quickly.

The system follows a layered client–server architecture made up of a mobile application, a backend server, a database and a notification service. It is organised into three main modules: the SOS Trigger Module, the Live Location Tracking Module, and the Contact and Notification Module. The user can register, log in, manage emergency contacts, trigger SOS, view her profile and mark herself safe. Alerts are delivered by push notification with SMS as a fallback, and the delivery status is tracked. The design prioritises the initial alert when the network is weak, tunes location polling to balance accuracy against battery and data use, and protects data through encrypted transmission and secure storage.

Testing under strong Wi-Fi, moderate 4G and weak mobile data showed that the alert reached trusted contacts within a few seconds under normal conditions and that tracking continued to update their view. The application aims to give quicker response and greater peace of mind to women and their families.

Index

S. No	Chapters	Topics	Page No
	Front matter	Declaration, Certificate, Acknowledgement, Abstract	2–5
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Importance of the Application 1.6 Target Users / Audience 1.7–1.12 Motivation, Literature Survey, Research Gap, Existing vs Proposed System, Methodology, Organisation	12
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks 2.4 Functional Requirements 2.5 Non-Functional Requirements	21
3	Technology Stack	3.1 Frontend 3.2 Backend 3.3 Database 3.4 APIs / Location Services 3.5 Authentication and Security 3.6 Development Environment 3.7 Version Control	28
4	System Architecture	4.1 High-Level Architecture 4.2 Architecture Components 4.3–4.6 Frontend, Backend, Database and Location Tracking Layers 4.7 Emergency Alert Flow 4.8 Deployment Architecture 4.9 Frontend–Backend Communication	33
5	System Design	5.1 System Design 5.2 Use Case Diagram 5.3 Data Flow Diagram 5.4 ER Diagram / Database Schema 5.5 Activity Diagram 5.6 Sequence Diagram 5.7 Class Diagram 5.8 User Interface Design	43
6	Implementation	6.1 Module-Wise Implementation 6.2–6.8 Account, Profile, Contacts, SOS, GPS, Tracking, Alert 6.9 Backend API Integration 6.10 Database Integration 6.11 Authentication and Authorization 6.12 Frontend Logic 6.13 Error Handling	62

S. No	Chapters	Topics	Page No
7	Features	7.1 Main Features 7.2–7.12 Feature-wise description 7.13 User Perspective 7.14 Technical Working	70
8	Testing	8.1 Introduction 8.2–8.9 Types of testing 8.10 Test Cases 8.11 Test Results 8.12 Error Handling 8.13 Performance Testing	76
9	Deployment	9.1 Introduction 9.2–9.5 Environment, Frontend, Backend, Database Setup 9.6 Environment Configuration 9.7 Running the Application 9.8 Localhost Configuration 9.9 Deployment Considerations	84
10	Challenges & Limitations	10.1 Challenges Faced 10.2–10.6 Specific Challenges 10.7 Solutions Applied 10.8 Current Limitations	89
11	Future Enhancements	11.1–11.7 Planned enhancements	93
12	Conclusion	12.1 Project Summary 12.2 Achievements 12.3 Benefits 12.4 Skills Learned 12.5 Final Conclusion	96
13	References	Reports, documentation and books	99
14	Appendices	Screenshots, sample code, installation, user manual, testing screenshots, review documentation	100

List of Figures

No.	Title	Page
4.1	System architecture of the Women Safety App (from the project review)	33
4.2	Layered view of the system architecture	34
4.3	Live location tracking flowchart	37
4.4	Emergency alert flow (SOS trigger to contact notification)	38
4.5	Prioritising the initial alert under weak connectivity	39
4.6	Deployment architecture	39
4.7	Frontend–backend communication	40
5.1	Module structure of the Women Safety App	43
5.2	Project flow of the Women Safety App (from the project review)	44
5.3	Use case diagram	45
5.4	Level 0 data flow diagram (context diagram)	46
5.5	Level 1 data flow diagram	47
5.6	Entity–relationship diagram	48
5.7	Activity diagram	50
5.8	Sequence diagram for the SOS alert	51
5.9	Sequence diagram for live tracking	52
5.10	Sequence diagram for login	53
5.11	Sequence diagram for emergency contact management	54
5.12	Class diagram	55
5.13	Welcome page	56
5.14	Login page	56
5.15	Registration page	57
5.16	Dashboard with SOS button	58
5.17	Emergency Contacts Page	58
14.1	Welcome page	100
14.2	Login page	100
14.3	Registration page	101
14.4	Dashboard with SOS button and feature cards	101
No.	Title	Page
14.5	Emergency Contacts Page	102
14.12	Sample Code – Backend SOS Alert Handler	104
14.13	Exact Installation Commands And Versions Used In The Project	105

14.17	Database Records Of SOS Event And Location Updates	108
14.18	Geo-Tag Photos With Guide (Dr. M Gayatri)	108
14.19	Review Forms With Guide Signatures	109

List of Tables

No.	Title	Page
1.1	User roles in the Women Safety App	17
1.2	Summary of the literature survey	18
1.3	Comparison of existing systems with the proposed system	19
2.1	Hardware requirements for the development machine	21
2.2	Hardware requirements for the end-user device	22
2.3	Hardware requirements for the server environment	22
2.4	Software requirements	23
2.5	Functional requirements	24
2.6	Non-functional requirements	25
2.7	Feasibility study	26
3.1	Technology stack of the Women Safety App	28
3.2	Summary of the technology choices and their purpose	32
4.1	Components of the architecture	34
4.2	Protection of data at each stage	40
4.3	Messages exchanged between the components	42
5.1	Description of the main use cases	45
5.2	Structure of the users table	48
5.3	Structure of the emergency_contacts table	48
5.4	Structure of the sos_events table	49
5.5	Structure of the location_updates table	49
5.6	Structure of the delivery_logs table	49
5.7	Interface guidelines followed in the design	61
6.1	Modules of the Women Safety App	62
6.2	Emergency contact fields	63
6.3	Backend request groups used by the application	66
6.4	States of an SOS event	67
6.5	Error situations and handling	68
6.6	Validation rules	68
7.1	Features and the objectives they satisfy	70
7.2	Chain of events during an emergency	74
8.1	Experimental setup	76
8.2	Test cases for registration and login	79
8.3	Test cases for emergency contacts	79

No.	Title	Page
8.4	Test cases for SOS, tracking and alerts	80
8.5	Summary of test results (as reported in the project review)	80
8.6	Performance evaluation	81
8.7	Test data used	82
8.8	Defect log format	83
9.1	Configuration settings required by the application	85
10.1	Challenges and solutions	89
10.2	Risk analysis	92
11.1	Future enhancements	93
11.2	Suggested order for the future enhancements	95
12.1	Objectives and their achievement	96
14.1	Abbreviations and terms used in this report	109
14.2	Requirements traceability matrix	110

CHAPTER – 1

INTRODUCTION

1.1 Background of the Project

Personal safety has become one of the most important concerns for women in modern society. Women and girls travel to colleges, workplaces, markets and railway stations every day, and many of them travel alone, late in the evening, or through isolated stretches of road. Incidents of harassment, stalking and assault in public spaces, on public transport and even in familiar neighbourhoods have made safety a daily worry instead of an occasional one. News reports and official crime records, such as the annual “Crime in India” publication of the National Crime Records Bureau, repeatedly show that the time taken to respond to an emergency is often the factor that separates a safe outcome from a serious tragedy.

When a woman feels threatened, the options available on an ordinary mobile phone are slow and clumsy. She must unlock the phone, open the dialler, select a number, wait for the call to connect and then explain where she is and what is happening, all while she may be frightened and under pressure. If the call is not answered, or if the other person cannot understand the situation quickly, valuable minutes are lost. A standard phone provides no dedicated single-action method to alert several trusted people at the same time with an exact location.

At the same time, almost every user already carries a smartphone with a built-in GPS receiver and an internet connection. This means that the hardware needed to solve the problem is already available in the user’s pocket; what is missing is simple and dependable software that uses it correctly. The Women Safety App – SOS and Live Tracking was conceived to fill this gap. It is a mobile application that allows a user to raise an alarm with a single tap. The moment the

SOS button is pressed, the application captures the real-time GPS coordinates of the device and sends an emergency alert to the trusted contacts that the user has already chosen. After the alert, the application keeps sharing the user's changing position with those contacts until the user marks herself safe or cancels the alert.

The project is developed as part of the II Year B.Tech (First Semester) curriculum of the Department of Computer Science and Information Technology (CSIT), K L University. It combines concepts from mobile application development, client–server architecture, database management, location-based services and notification mechanisms. The application is designed around a layered client–server architecture in which the mobile application acts as the client, a backend server processes alerts and location updates, a database stores user, contact and event data, and a notification service delivers alerts to the guardians of the user.

1.2 Problem Statement

Women and girls who travel alone, commute late at night or move through isolated areas face a real and persistent risk of harassment, stalking and assault. When an emergency occurs, the existing options are slow and clumsy: dialling a number, explaining the situation under panic, and hoping that the call connects and is understood in time. Standard mobile phones offer no dedicated, single-action way to alert multiple trusted people at once with an exact location. Family members are frequently unaware of the whereabouts of a loved one during a commute, and by the time help is arranged, precious minutes have already been lost.

There is therefore a clear need for a lightweight mobile application that allows a woman in distress to raise an alarm instantly with one tap, and that keeps her trusted circle continuously

informed of her live location until she is safe. The problem addressed by this project can be summarised in the following points:

- The time required to call and explain an emergency is too long when the user is under threat.
- A phone call reaches only one person at a time, and it gives no automatic location information.
- A single shared location shows only where the emergency began and not where the user is moving afterwards.
- Many safety solutions require several taps or menu navigation, which is unrealistic during panic.
- Emergencies often happen where the mobile network is weak, yet many applications expect a stable internet connection.

1.3 Scope of the Project

The scope of the project is limited to the design and development of a mobile safety application whose core purpose is to raise an instant SOS alert and to share live location with trusted contacts. The following functions are included within the scope of the project:

- User registration and login using name, mobile number, email address and password.
- A dashboard that gives access to the SOS button, Emergency Contacts, Live Tracking and Profile.
- Adding, editing and removing emergency contacts (trusted guardians).

- A one-tap SOS trigger that captures the current GPS coordinates and sends an alert request to the backend server.
- Continuous live location tracking after the SOS is raised, with updates sent to the backend at regular short intervals.
- Delivery of alert notifications, including a live location link, to emergency contacts through a push-notification service, with SMS as a fallback.
- Tracking of delivery status for each contact so that the user's device can confirm that the guardians have received the alert.
- Storage of user profiles, emergency contacts, SOS events and location history in a database, with encrypted transmission and secure storage.
- Marking the user as safe or cancelling the alert, which stops the tracking.

The project does not claim to replace the police, medical or other official emergency services. Direct integration with police control rooms, voice-activated SOS, offline alert queuing, wearable devices and machine-learning-based anomaly detection are identified as future enhancements and are discussed in Chapter 11. They are not part of the present implementation.

1.4 Objectives of the Project

The main aim of the project is to reduce the time between the moment a threat is felt and the moment help is on the way, by replacing multiple manual actions with a single reliable trigger.

The specific objectives of the project are as follows:

1. To design a single-tap SOS mechanism that instantly alerts pre-selected trusted contacts during an emergency.

2. To implement continuous live GPS location tracking that guardians can follow until the situation is resolved.
3. To build a simple interface that works reliably with minimal interaction under high-stress conditions.
4. To provide secure management of emergency contacts, including adding, editing and removing trusted guardians.
5. To ensure fast and dependable alert delivery even under limited or unstable network connectivity.
6. To protect user data and location history through secure storage and a privacy-conscious design.

In addition to these functional objectives, the project has an academic objective. It gives the team practical experience of the complete software development life cycle, beginning with requirement analysis and system design, followed by module development, integration, testing and documentation.

1.5 Importance of the Application

The importance of the Women Safety App lies in the speed and simplicity with which it can act in a critical situation. In a crisis, every additional step increases the delay. By offering a single prominent SOS button with no confirmation dialogs or multi-step menus, the application removes the need for the user to think, search or explain. The alert, with the exact location, reaches several trusted people at the same moment.

The second source of importance is continuity. Most applications of this kind stop after sending one location message. In real emergencies, however, the user is often moving, for example inside a vehicle or while walking away from danger. Continuous live tracking lets family members and responders follow the actual movement of the user, such as the route being taken towards home, instead of seeing only the point at which the emergency started. This improves transparency and helps guardians decide quickly on a suitable response.

The third source of importance is awareness and peace of mind. When family members know that they will be notified instantly if something goes wrong, they feel more confident about allowing the user to travel independently, and the user herself feels more secure. The application also makes use of hardware that people already own, so it does not require any additional device or cost for the user. The project also shows how widely available technology can be used for a social purpose, which gives it value beyond the academic requirement.

1.6 Target Users / Audience

The primary users of the application are women and girls who travel alone or who move through areas where they feel unsafe. This includes college students, working women who commute at night, women who use public transport, and those who walk through isolated roads or unfamiliar areas. The secondary users are the trusted contacts of these women. They are usually parents, siblings, spouses, relatives and close friends who receive the alert and follow the live location. Their role is passive but essential, since the usefulness of the application depends on how quickly they receive and act on the alert.

Table 1.1: User roles in the Women Safety App

Role	Who they are	Interaction with the system
User (primary)	A woman who has registered in the app	Registers, logs in, manages emergency contacts, presses the SOS button, shares live location, marks herself safe, views her profile and logs out.
Emergency contact (guardian)	Family member or trusted friend added by the user	Receives the SOS alert with the live location link, tracks the movement of the user and takes action or responds.
Backend server	System component	Authenticates users, processes SOS requests, stores events and location updates and triggers notifications.

1.7 Motivation

The motivation for this project comes from the rising number of crimes against women in public places and the observation that delayed response is one of the biggest factors in the outcome. Technology can meaningfully shrink the gap between the time a threat is felt and the time help starts moving. The team was motivated by the belief that a simple, dependable piece of software, installed on a device the user already owns, can play a real part in this. Rather than designing a complex product with many features, the team decided to concentrate on two things that matter most in an emergency: an instant trigger, and a continuous, reliable flow of information to the people who can help.

1.8 Literature Survey

Before designing the system, the team studied existing work in five areas. The findings of this survey are summarised in Table 1.2.

Table 1.2: Summary of the literature survey

Area studied	What was observed	Influence on this project
Existing safety apps	Commercial applications such as bSafe and similar SOS tools were studied. Most provide a one-time location share rather than a continuously updated live trail.	Motivated continuous live tracking after the first alert.
Panic-button research	Academic studies on panic-button systems show that reducing the number of taps needed to raise an alarm significantly improves usability in a crisis.	Led to the single-button, no-confirmation SOS design.
GPS tracking systems	Prior work on fleet and child-tracking applications informed continuous location sharing.	Guided the design of the live tracking module.
Push-notification architectures	Literature on real-time messaging, for example systems based on Firebase Cloud Messaging, guided the choice of a lightweight backend for instant alerts.	Guided the notification design and the use of a push-notification service.
Gap identified	None of the surveyed systems combined one-tap SOS, continuous live tracking and low-connectivity reliability in one seamless application.	Defined the research gap addressed by this project.

1.9 Research Gap

The literature survey shows that several individual pieces of the safety problem have already been addressed, but no single application combines them well. Most SOS applications stop after sending one location update and then go silent, so guardians cannot follow the real movement of the user. Systems that do offer continuous tracking are often built for logistics or parental control and not for a panic situation, so they need too many steps to activate quickly. Very few solutions are designed to keep working when the network is weak, which is exactly the situation in which many real emergencies occur. Finally, little attention is paid to keeping the interaction minimal, since most applications still expect the user to navigate menus instead of reacting instinctively.

This project addresses the gap by combining a true one-tap trigger, continuous live tracking and connectivity-aware alert delivery in a single purpose-built mobile application.

1.10 Existing System versus Proposed System

The comparison between the commonly used existing methods and the proposed Women Safety App is shown in Table 1.3.

Table 1.3: Comparison of existing systems with the proposed system

Feature	Existing systems	Proposed Women Safety App
Alert method	Mostly manual calls or messages	Single-tap automated SOS trigger
Location sharing	Typically a one-time location	Continuous live tracking until the situation is resolved
Interaction required	Often multiple taps or steps	Minimal interaction designed for high-stress use
Connectivity handling	Many apps depend on a stable internet connection	Prioritises delivery of the initial alert even under weak connectivity
Contact management	Rarely integrated with location sharing	Contact management and location sharing combined in one flow

1.11 Proposed Methodology

The project was carried out in six stages, which together follow a simple iterative life cycle.

1. **Requirement analysis:** existing safety solutions were studied, and the functional and non-functional requirements of the SOS and tracking features were identified.
2. **System design:** a layered client–server architecture was designed, separating the mobile application, the communication layer, the backend server and the database.
3. **Module development:** the SOS trigger, live tracking and contact management modules were built independently and then integrated into one application.
4. **Location and alerting:** device GPS/location services are used on the client side, and a push-notification service is used on the backend to deliver near-instant alerts.

5. **Testing:** each module was validated under normal and low-connectivity conditions, and the response time from trigger to contact notification was observed.
6. **Refinement:** the interface and performance were improved on the basis of test results so that the interaction stays minimal and dependable under stress.

1.12 Organisation of the Report

This report is divided into fourteen chapters. Chapter 1 introduces the project. Chapter 2 lists the hardware and software requirements, and Chapter 3 describes the technology stack. Chapter 4 explains the system architecture, and Chapter 5 presents the system design through diagrams and the database schema. Chapter 6 describes the implementation module by module. Chapter 7 explains the features from the user's perspective and the technical perspective. Chapter 8 covers testing, and Chapter 9 describes setup and deployment. Chapter 10 discusses the challenges and limitations, Chapter 11 proposes future enhancements, and Chapter 12 concludes the report. Chapter 13 lists the references, and Chapter 14 contains the appendices.

CHAPTER – 2

SYSTEM REQUIREMENTS

System requirements describe the hardware and software resources that are needed to develop, test and run the Women Safety App. Because the application has two parts, a mobile client used by the user and a server side that processes alerts, the requirements are described separately for the development environment, the end-user device and the server environment. The requirements given below are the minimum and recommended configurations for a project of this size, and they follow the experimental setup used during testing, in which mid-range Android smartphones with GPS enabled were used as typical end-user devices.

2.1 Hardware Requirements

2.1.1 Development Machine

The development machine is used to write the application code, run the backend server locally and manage the database. A machine with moderate specifications is sufficient, since the application is lightweight, but additional memory improves the speed of the build tools and of any emulator used during testing.

Table 2.1: Hardware requirements for the development machine

Component	Minimum	Recommended
Processor	Dual-core processor, 2.0 GHz	Quad-core processor, 2.5 GHz or higher
RAM	4 GB	8 GB or more
Storage	20 GB free disk space	50 GB free space on an SSD
Display	1366 × 768 resolution	1920 × 1080 resolution
Network	Broadband or Wi-Fi connection	Stable broadband or Wi-Fi connection
Peripherals	Keyboard and mouse	Keyboard, mouse and USB cable for connecting a test phone

2.1.2 End-User Device

The end-user device is the smartphone on which the application runs. The most important hardware element is the GPS receiver, because the entire SOS and tracking function depends on it. A data connection is needed to send the alert and the location updates to the backend server.

Table 2.2: Hardware requirements for the end-user device

Component	Requirement
Device type	Android smartphone (mid-range devices were used for testing)
GPS / location hardware	Built-in GPS receiver with location services enabled
Network	Wi-Fi or mobile data (4G); the application is designed to keep working under weak or intermittent connectivity
Battery	Sufficient charge, since continuous GPS tracking consumes power
Display and input	Touch screen large enough to show one prominent SOS button

2.1.3 Server Environment

The backend server receives the SOS alert and the location updates, stores them in the database and triggers the notifications. During development the server runs on the same machine as the development tools. For wider use it can run on any computer or hosted server that has a stable network connection, sufficient memory to handle simultaneous alerts and enough storage for the database.

Table 2.3: Hardware requirements for the server environment

Component	Minimum	Recommended
Processor	Dual-core processor	Quad-core processor
RAM	4 GB	8 GB or more
Storage	20 GB	50 GB or more, depending on location history retained
Network	Reliable internet connection	Stable connection with a fixed address reachable from the mobile app

2.2 Software Requirements

The software requirements are grouped into operating systems, the mobile platform, the server software, the database and supporting services. The exact tool names that the team used for each layer are listed in Chapter 3, and the entries are marked there so that they can be confirmed before final submission.

Table 2.4: Software requirements

Category	Requirement
Development operating system	Windows, Linux or macOS on the development machine
Mobile operating system	Android on the test and end-user devices
Mobile application development environment	An Android/iOS mobile application development framework with an IDE and emulator support
Server-side runtime	A server-side application environment that handles authentication, alert processing and location updates
Database	Cloud or relational database for user profiles, emergency contacts and location history
Location services	Device GPS and a mapping / location API for capturing and showing real-time coordinates
Notification services	Push-notification service for instant alerts, with SMS as fallback
Security	Encrypted data transmission and secure storage
Testing tools	API testing tool, emulator and physical device for testing under different network conditions
Documentation tools	Word processor, diagram tools and a presentation tool for the project review

2.3 Development Tools and Frameworks

The team used a set of tools that support every stage of development, from design to testing. These are described by their role in the project.

2.3.1 Mobile Development Environment

A mobile application development framework and its integrated development environment are used to build the screens of the application: the welcome page, registration page, login page,

dashboard, contacts page, live tracking page and profile page. The environment provides a code editor, a layout designer, a debugger, and an emulator that imitates a smartphone. Since GPS behaviour is hard to judge on an emulator alone, testing was also done on physical Android smartphones.

2.3.2 Backend Development Tools

The backend is developed with a server-side framework that exposes request handlers for registration, login, contact management, SOS alerts and location updates. An API testing tool is used to send sample requests to the backend and to check the responses before the mobile application is connected to it.

2.3.3 Database Tools

A database management tool is used to create the tables, insert test records and run queries to verify that user details, contacts, SOS events and location updates are stored correctly.

2.3.4 Design and Documentation Tools

Diagramming tools were used for the architecture diagram, data flow diagrams, ER diagram and other figures in this report. The user interface was designed with a consistent pink and purple colour theme and a single large SOS button on the dashboard so that the most important action is always obvious.

2.4 Functional Requirements

Table 2.5: Functional requirements

ID	Requirement	Description
FR1	User registration	The system shall allow a new user to register using name, mobile number, email address and password.
FR2	User login	The system shall allow a registered user to log in with email or phone number and password.
FR3	Dashboard	The system shall display a dashboard with the SOS button and access to Emergency Contacts, Live Tracking and Profile.

ID	Requirement	Description
FR4	Contact management	The system shall allow the user to add, edit and remove emergency contacts.
FR5	One-tap SOS	The system shall raise an SOS alert with a single tap and no confirmation step.
FR6	Location capture	The system shall capture the real-time GPS coordinates of the device when SOS is pressed.
FR7	Alert delivery	The system shall send the alert with a live location link to all emergency contacts.
FR8	Fallback delivery	The system shall use SMS where push delivery is not possible.
FR9	Live tracking	The system shall send location updates at regular intervals until the alert ends.
FR10	Delivery status	The system shall track the delivery status for each contact.
FR11	Stop SOS	The system shall stop tracking when the user marks herself safe or cancels the alert.
FR12	Profile	The system shall let the user view her account details.
FR13	Logout	The system shall let the user log out of the application.

2.5 Non-Functional Requirements

Table 2.6: Non-functional requirements

Attribute	Requirement
Usability	The interface must work with minimal interaction; the SOS action needs only one tap, with no confirmation dialog or menu.
Performance	The alert should reach the contacts within a few seconds under normal network conditions.
Reliability	Alert delivery must stay consistent when several contacts are notified at the same time.
Connectivity	The initial alert must be prioritised over location streaming when bandwidth is limited.
Security and privacy	Data must be transmitted in encrypted form and stored securely; live location history is held for a limited time.
Efficiency	Location polling intervals must balance freshness against battery and data consumption.
Availability	Tracking must continue in the background if the screen is locked or another app is opened.
Maintainability	The system must be built in independent modules so that each can be modified without affecting the others.

2.6 Feasibility Study

Before development began, the team examined whether the project could be carried out successfully with the resources available. The study considered four aspects.

Table 2.7: Feasibility study

Aspect	Assessment
Technical feasibility	The project depends on a smartphone with GPS, a data connection, a backend server, a database and standard notification services. All of these are widely available, and the required skills in mobile development, server-side programming and databases are covered by the II Year B.Tech curriculum. The project is therefore technically feasible.
Economic feasibility	The user needs no extra device because the application uses the smartphone she already owns. Development tools used for a project of this size are freely available to students. Costs would arise only if the system were deployed for many users, for example for hosting and SMS delivery.
Operational feasibility	The interface is made of a few simple screens with one prominent SOS button, so a user needs almost no training. Emergency contacts need no special setup to receive the alert.
Legal and privacy aspects	The application handles personal data and location. It therefore uses encrypted transmission, secure storage and time-limited retention of location history, and shares location with the contacts chosen by the user only.

2.7 Assumptions and Constraints

The design and testing of the project rest on some assumptions, and several constraints limit what the present version can do.

2.7.1 Assumptions

- The user owns a smartphone with a working GPS receiver and has granted the location permission to the application.
- The user has added at least one emergency contact before using the SOS function.
- The user has at least some mobile data or Wi-Fi connectivity at the time of the alert, even if it is weak.

- The emergency contacts have phones that can receive push notifications or SMS messages.

2.7.2 Constraints

- The quality of the location depends on the GPS signal and on the device.
- Continuous tracking is limited by the battery and data capacity of the device.
- The SOS can be triggered only through the on-screen button in the present version.
- The alert is delivered to personal contacts and not directly to official emergency services.

CHAPTER – 3

TECHNOLOGY STACK

The technology stack of the Women Safety App follows the layered architecture shown on the system architecture slide of the project review. It consists of a mobile frontend, location services, a backend server, a database, a notification service and a security layer. This chapter describes the purpose of each layer and the way it is used in the project.

Note for the team (delete before submission): The project review presentation describes the technologies by category and does not name specific products. The technology names in Table 3.1 are therefore written in general terms. Wherever you see a yellow highlighted entry, replace it with the exact framework, language, database or service that you actually used, and keep the same names in every chapter.

Table 3.1: Technology stack of the Women Safety App

Layer	Role in the project	Technology used
Frontend	Android/iOS mobile application with SOS button, contacts, tracking and profile screens	[Specify mobile framework and language]
Location services	Device GPS and mapping/location APIs for capturing and streaming real-time coordinates	[Specify location / maps API]
Backend	Server-side application for authentication, alert processing and location updates	[Specify backend framework and language]
Database	Cloud/relational database for user profiles, contacts and location history	[Specify database]
Notifications	Push-notification service for instant SOS alerts, with SMS fallback	[Specify push service and SMS provider]
Security	Encrypted transmission and secure storage	[Specify method, e.g. HTTPS and password protection]

3.1 Frontend

The frontend is the part of the system that the user actually sees and touches. In this project it is a mobile application built with an Android/iOS mobile application development framework.

The frontend is responsible for showing the welcome page, registration and login forms, the

dashboard, the contact management screen, the live tracking map and the profile page. It is also responsible for obtaining the GPS position from the device and for sending the SOS alert and location updates to the backend.

The frontend was designed with one principle in mind: during an emergency the user should not have to think. For this reason the dashboard shows a large red SOS button at the centre of the screen, and below it three simple cards labelled Emergency Contacts, Live Tracking and Profile. There are no confirmation dialogs or nested menus between the user and the alert. The screens follow a consistent pink and purple colour scheme, with rounded buttons and clear labels, which was visible in the screenshots taken during the project review.

The frontend also handles the client side of the live tracking module. After an alert is sent, it reads the GPS position at short regular intervals and passes each reading to the backend. This work continues in the background even when the screen is locked or another application is opened, which is important because a user in danger will rarely keep the application in the foreground.

3.2 Backend

The backend is a server-side application that sits between the mobile app and the database. It receives requests from the app over a secure connection and performs the following tasks: it registers new users and checks login credentials, stores and retrieves emergency contacts, receives the SOS alert and the location updates, writes events to the database, and asks the notification service to inform the emergency contacts.

The backend is designed as a set of small independent services: an authentication service, an SOS alert processor, a location update service, and a contact and notification service. Keeping

them separate makes the code easier to test, and it means a fault in one service does not stop the others. The SOS alert processor is intentionally given the highest priority, because the initial alert is the most important message in the whole system.

3.3 Database

The database stores the information the system needs to function and to be evaluated. The project review identifies four groups of data: user profile data, emergency contact data, location data and event data. These are stored in a cloud or relational database in the form of tables for users, emergency contacts, SOS events, location updates and delivery logs. The detailed schema is described in Section 5.4.

A relational design suits this project because the data is naturally connected. A user has many contacts, a user can raise many SOS events, each SOS event has many location updates, and each event generates delivery records for the contacts. Relationships and keys keep this data consistent, and queries such as “fetch all contacts of this user” or “fetch the latest location of this SOS event” are simple and fast.

3.4 APIs / Location Services

Two kinds of external services are used. The first is the location service of the device, which provides real-time latitude and longitude from the GPS receiver. The second is a mapping and location API, which is used to display the live position on a map and to build the live location link that is sent to the contacts. The literature survey mentioned Google Maps Platform and OpenStreetMap as examples of such services, and the Android developer documentation on location and context was used as a reference for the client side.

The notification service is the third external element. A push-notification service delivers the alert quickly to the smartphones of the guardians. Push delivery is not possible in every case, for example when the guardian's phone is offline or the app is not installed, and so SMS is used as a fallback. The status of each delivery is recorded and returned so that the user's device can confirm that the alert has actually reached the guardians.

3.5 Authentication and Security

The application handles sensitive information: a person's identity, her phone numbers, the numbers of her trusted contacts and, above all, her real-time location. For this reason security is treated as a requirement and not as an optional feature. The project uses the following measures.

- **Authenticated access:** a user must register and log in with credentials before any feature can be used.
- **Encrypted transmission:** all communication between the mobile app and the backend is carried over an encrypted connection, so that location and contact data cannot be read in transit.
- **Secure storage:** user data, contacts and location history are stored securely on the backend.
- **Time-limited location history:** live location history is stored for a limited time only, which reduces the privacy risk.
- **Minimal exposure to contacts:** guardians receive a live location link and not unrestricted access to the user's data.

3.6 Development Environment

The development environment consists of a computer running a standard desktop operating system, the mobile application development tools with an emulator, a local copy of the backend server and a database instance. Physical Android smartphones with GPS enabled are connected for testing, because real GPS behaviour and real network conditions cannot be reproduced fully on an emulator. During development the backend runs locally, and the mobile application is configured to connect to it. Later chapters describe this configuration in more detail.

3.7 Version Control

Version control is a basic requirement for any team project. It records every change made to the source code, allows team members to work on different modules at the same time and makes it possible to return to an earlier working version when a change introduces a fault. In this project three students worked on the SOS trigger, live tracking and contact modules independently and then integrated them, which is the kind of work that version control supports. **[Confirm and state the system used, for example Git with a GitHub repository, and the repository name].**

Table 3.2: Summary of the technology choices and their purpose

Technology area	Why it is needed	Benefit to the project
Mobile application framework	To build one interface for SOS, contacts, tracking and profile	Single-tap access; background operation
GPS / location API	To obtain real-time coordinates and show them on a map	Accurate, live location for guardians
Backend server	To process alerts and location updates centrally	Fast alert processing and contact notification
Relational / cloud database	To store users, contacts, events and location history	Consistent, queryable data
Push notification service and SMS	To reach guardians instantly	Reliable delivery with fallback

Technology area	Why it is needed	Benefit to the project
Encryption and secure storage	To protect location and contact data	Privacy-conscious design

CHAPTER – 4

SYSTEM ARCHITECTURE

The system architecture describes how the major parts of the Women Safety App are organised and how they communicate with each other. A clear architecture is especially important in a safety application, because the path from the user's tap to the notification on a guardian's phone must be short, predictable and free from unnecessary steps.

4.1 High-Level Architecture

The Women Safety App follows a layered client–server architecture. The user interacts with the mobile application, which is the frontend. The mobile application obtains the real-time position from the GPS / location service of the device and sends an API request containing the location and the alert to the backend server. The backend server receives the SOS and the location, stores the data in the database, sends notifications and keeps the live location updated. The emergency contacts, who are the guardians and family members of the user, receive the alert message and a live location link through which they can track the movement of the user and take action.

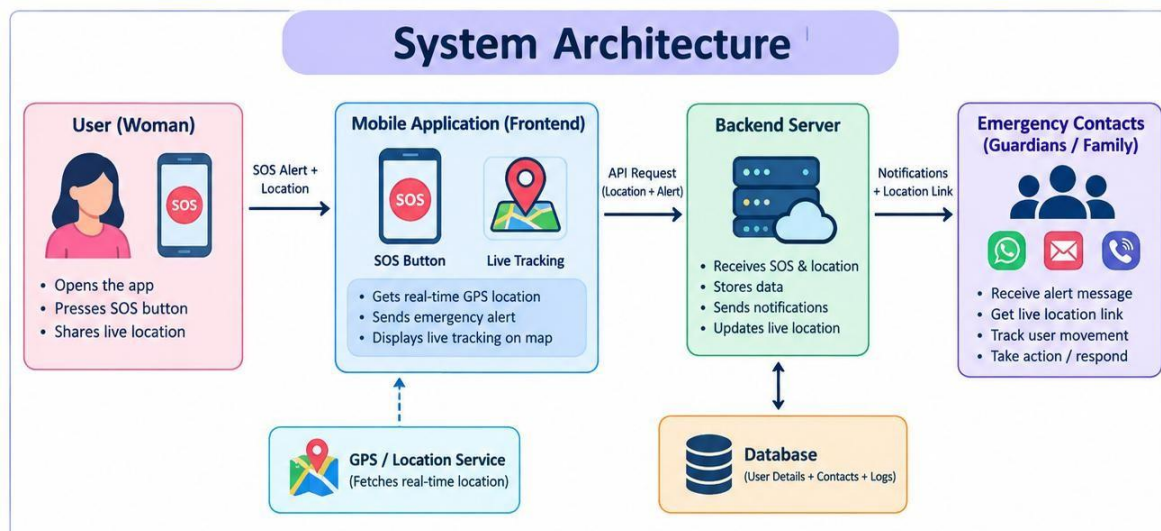


Figure 4.1: System architecture of the Women Safety App (from the project review)

The same architecture is redrawn as a layered diagram in Figure 4.2. The diagram groups the components into a presentation layer, an application layer, a data layer and a group of external services.

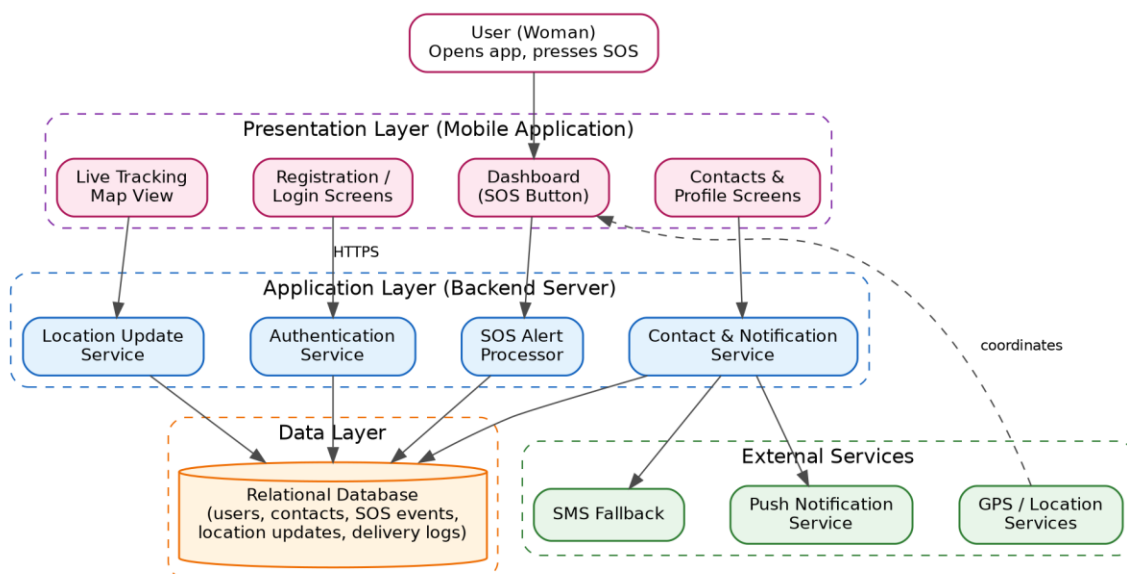


Figure 4.2: Layered view of the system architecture

4.2 Architecture Components

Table 4.1: Components of the architecture

Component	Responsibility	Communicates with
User	Opens the app, presses the SOS button and shares live location	Mobile application

Component	Responsibility	Communicates with
Mobile application (frontend)	Gets real-time GPS location, sends the emergency alert and displays live tracking on a map	GPS service, backend server
GPS / location service	Fetches the real-time location of the device	Mobile application
Backend server	Receives SOS and location, stores data, sends notifications and updates live location	Mobile app, database, notification service
Database	Stores user details, contacts and logs	Backend server
Notification service	Delivers the alert and live location link by push, with SMS fallback	Backend server, emergency contacts
Emergency contacts	Receive the alert, open the live location link, track the user and respond	Notification service

4.3 Frontend Layer

The frontend layer is the mobile application running on the user's smartphone. It contains the screens that were shown in the project review: the welcome page with the Get Started button, the registration form, the login form and the dashboard. The dashboard is the centre of the application. From it the user can press the SOS button or move to Emergency Contacts, Live Tracking and Profile.

Apart from presenting screens, the frontend layer performs three important tasks. First, it validates what the user types, for example by checking that required fields are filled before a registration request is sent. Second, it calls the location service of the device and reads the coordinates. Third, it communicates with the backend through requests that carry the user's identity along with the data. To keep the SOS trigger fast, this layer does not ask the user for confirmation and it sends the alert request as soon as the coordinates are available.

4.4 Backend Layer

The backend layer is the brain of the system. When a request arrives, it first identifies the user and then passes the request to the appropriate service. The authentication service handles

registration and login. The SOS alert processor receives the alert, records the SOS event and starts the notification process. The location update service stores each new coordinate pair and keeps the live view of the contacts current. The contact and notification service fetches the contact list and dispatches notifications.

The backend also performs two optimisations described in the project review. It throttles and deduplicates location updates sent to contacts, which keeps notifications timely without overwhelming the guardians' devices. And when bandwidth is limited, it gives priority to the initial SOS alert over the continuous tracking data, so that the guardians are notified first.

4.5 Database Layer

The database layer stores all persistent data. It holds four groups of data that were identified in the project: user profile data (name, phone number and login credentials), emergency contact data (names and phone numbers of guardians), location data (latitude and longitude streamed during an active alert) and event data (timestamped SOS trigger events, alert status and delivery confirmations). Only the backend server talks to the database. The mobile application never accesses it directly, which keeps the data protected and gives the backend full control over validation.

4.6 Location Tracking Layer

The location tracking layer consists of the GPS / location service on the device and the live tracking logic in the application. When the SOS is triggered, the current position is captured once and sent with the alert. After that, the live tracking module reads the position at short, regular intervals and streams the updates to the backend. The polling interval is tuned to balance

location accuracy against battery and data usage. Tracking stops automatically when the user marks herself safe or cancels the alert.

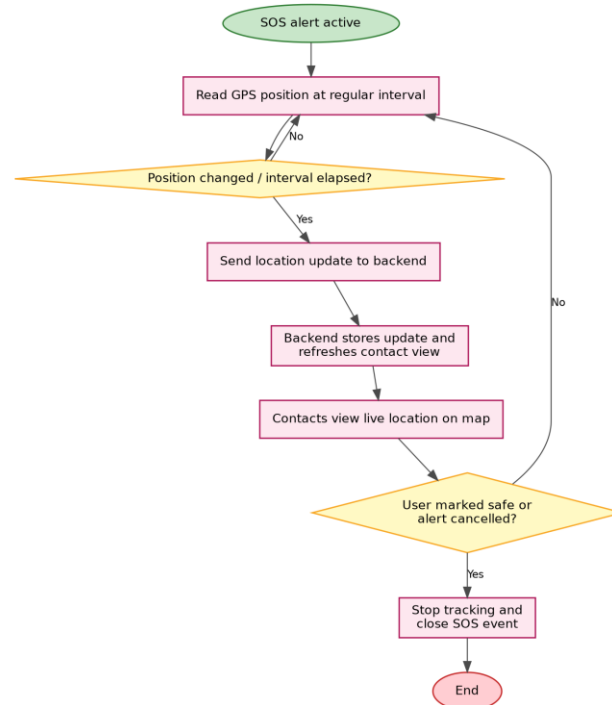


Figure 4.3: Live location tracking flowchart

4.7 Emergency Alert Flow

The emergency alert flow describes what happens from the moment the SOS button is pressed until the guardians have been notified. It is the most important sequence in the application and is shown in Figure 4.4.

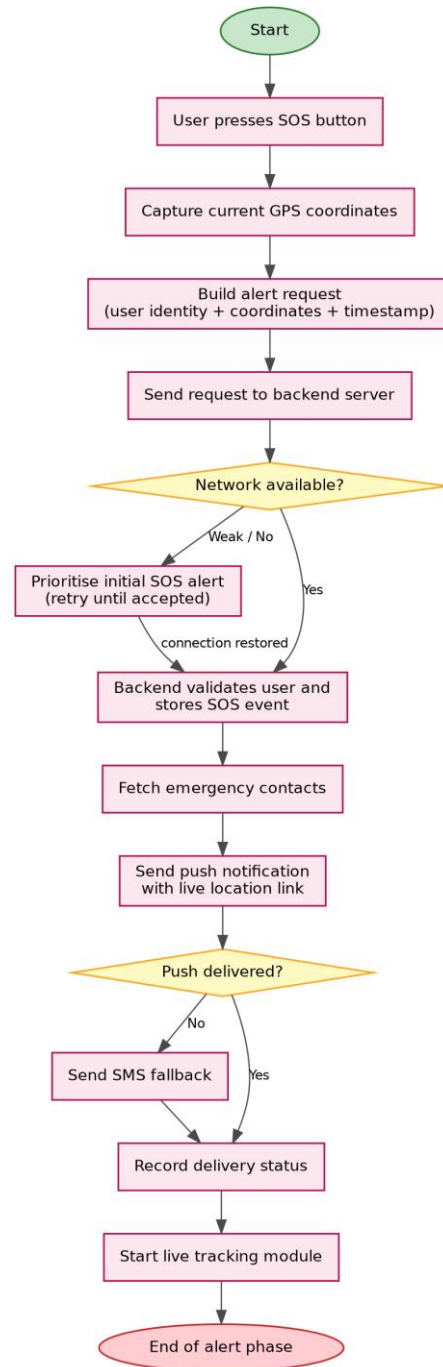


Figure 4.4: Emergency alert flow (SOS trigger to contact notification)

The sequence is as follows. The user presses the SOS button. The application captures the GPS coordinates, builds an alert request containing the identity of the user, the coordinates and the time, and sends it to the backend. If the network is weak, the initial alert is given priority and is retried until it is accepted. The backend validates the user, stores the SOS event, retrieves the emergency contacts and sends a push notification that includes the live location link. If

push delivery fails, an SMS is sent as a fallback. The delivery status of every contact is recorded. Finally, control passes to the live tracking module so that location sharing continues without any further action by the user.

4.7.1 Handling of Low Connectivity

Many real emergencies happen where the network is weak. The alert pipeline therefore treats the initial SOS message as the most important data and sends it first with a small payload. Continuous location streaming is held back while the connection is weak and resumes automatically once the connection is stable. This ensures that the guardians are notified even if live tracking has to wait for a short time.



Figure 4.5: Prioritising the initial alert under weak connectivity

4.8 Deployment Architecture

The deployment architecture shows where each part of the system runs. The mobile application is installed on the user's Android smartphone. It communicates over Wi-Fi or mobile data, using an encrypted connection, with the backend server. The backend server is connected to the database server and to the notification services. The notification services deliver the alert to the smartphones of the guardians by push notification or by SMS.

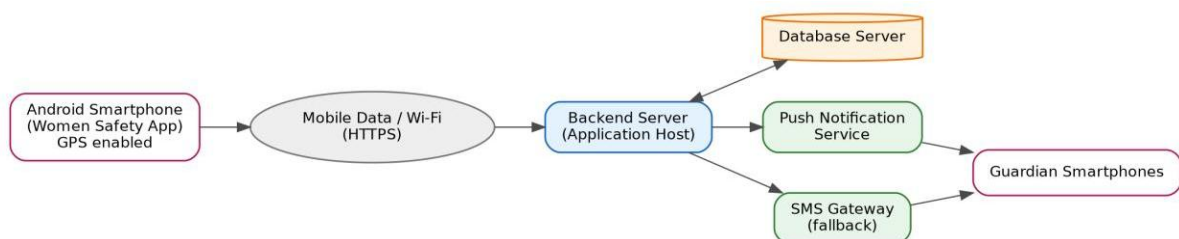


Figure 4.6: Deployment architecture

During development and testing, the backend server and the database run on a single development machine and the mobile application connects to it over the local network. For wider use, the same components can be placed on a hosted server with a fixed address. Chapter 9 describes the setup and the considerations for such a deployment.

4.9 Frontend–Backend Communication

The mobile application and the backend communicate through request–response messages. Each user action that needs server processing is turned into one request and the backend answers with a result. The main exchanges are shown in Figure 4.7.

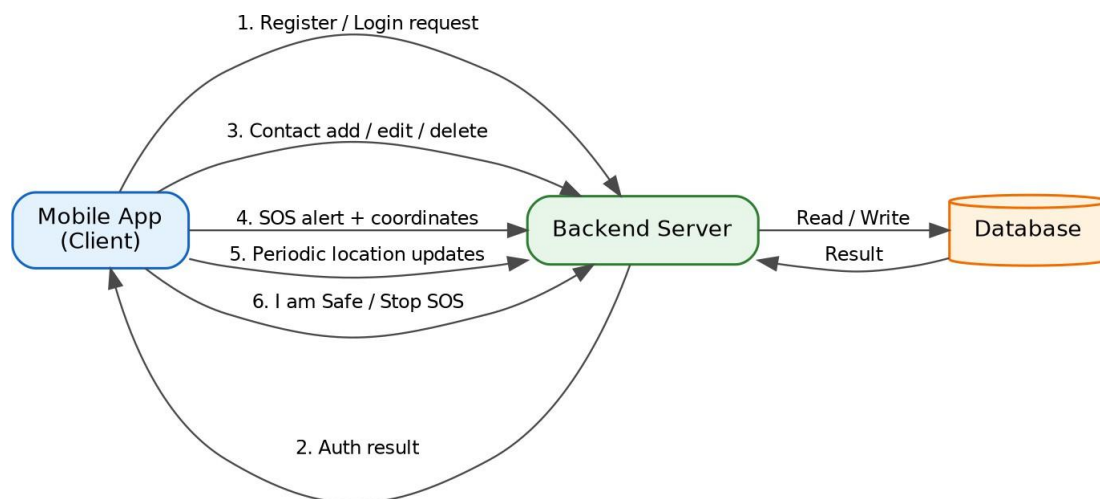


Figure 4.7: Frontend–backend communication

4.10 Data Security Architecture

Security is built into the architecture and is not added afterwards. Data passes through the system in three stages: on the device, in transit and at rest on the server. The protection applied at each stage is shown in Table 4.2.

Table 4.2: Protection of data at each stage

Stage	Data involved	Protection
On the device	Login details entered by the user; GPS coordinates	Data is collected only for the functions that need it; location is read only when needed for SOS and tracking
In transit	Registration, login, contact, SOS and location requests	Encrypted transmission between the application and the backend
At rest on the server	User profiles, contacts, SOS events, location history, delivery logs	Secure storage; limited access through the backend only; time-limited retention of live location history
At the guardian	Live location link	The guardian receives a link tied to the SOS event instead of access to the account

This arrangement ensures that the sensitive information reaches only those for whom it is intended, and that it does not remain available for longer than the emergency requires.

4.11 Design Principles

The architecture follows a small number of principles that were kept in mind at every stage of the project.

- **Speed first:** the path from the SOS tap to the notification is kept as short as possible, with no confirmation steps.
- **Priority for the first alert:** the initial SOS message always has priority over continuous tracking data.
- **Separation of concerns:** the application, the backend and the database are separate layers, and the modules within them are independent.
- **Fail visibly:** when something goes wrong, for example a missing permission, the user is told, so that she does not assume that an alert has been sent.

- **Privacy by design:** data is encrypted in transit, stored securely, and kept for a limited time.
- **Minimal interaction:** the interface offers one large button for the emergency and keeps everything else out of the way.

4.12 Data Exchanged between Components

Each connection in the architecture carries a specific piece of information. Knowing exactly what is sent in each message helps in testing and also in judging the privacy of the system. The principal messages are listed in Table 4.3.

Table 4.3: Messages exchanged between the components

Message	From → To	Contents	Purpose
Registration request	App → Backend	Name, mobile number, email, password	Create a user account
Login request	App → Backend	Email or phone number, password	Authenticate the user
Contact request	App → Backend	User identity, contact name, phone number	Add, edit or delete a contact
SOS alert request	App → Backend	User identity, latitude, longitude, time	Raise the emergency alert
Location update	App → Backend	User identity, latitude, longitude, time	Keep the live position current
Alert notification	Backend → Contact	Alert message and live location link	Inform the guardian
Delivery status	Notification service → Backend → App	Status for each contact	Confirm that guardians received the alert
Stop SOS request	App → Backend	User identity, event identity	End tracking and close the event

The alert request is deliberately small. Because it contains little more than the identity of the user, the coordinates and the time, it can be sent quickly even on a weak connection, which supports the design rule that the first alert has priority over everything else.

CHAPTER – 5

SYSTEM DESIGN

5.1 System Design

System design converts the requirements of Chapter 2 into a blueprint that can be implemented. The design of the Women Safety App is based on a modular approach. The application is divided into five modules, each with a clear responsibility: the User Account module, the Contact and Notification module, the SOS Trigger module, the Live Tracking module and the Profile and Secure Storage module. The SOS Trigger module is the entry point of the application. When it completes its work it hands control to the Live Tracking module, and both modules depend on the Contact and Notification module for delivery of messages.

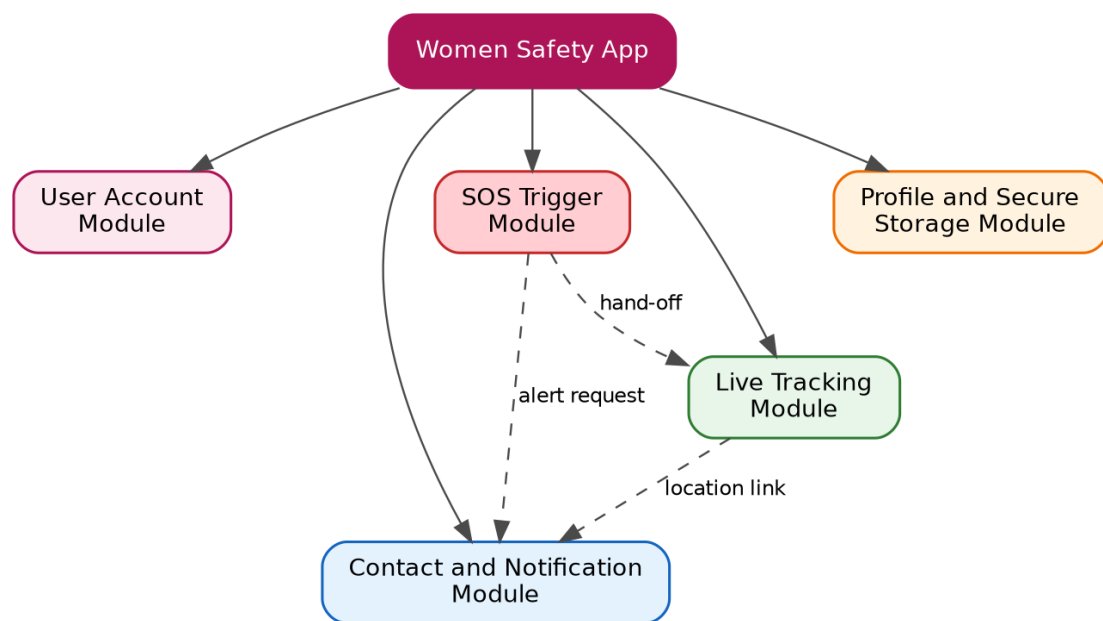


Figure 5.1: Module structure of the Women Safety App

Figure 5.2 shows the typical flow of use as presented in the project review. The user opens the app and reaches the login or registration page. After logging in, the user reaches the dashboard, where all features are available. On pressing SOS, the real-time location is captured and the

alert is sent to trusted contacts. The contacts can then track the live location until the emergency is resolved.

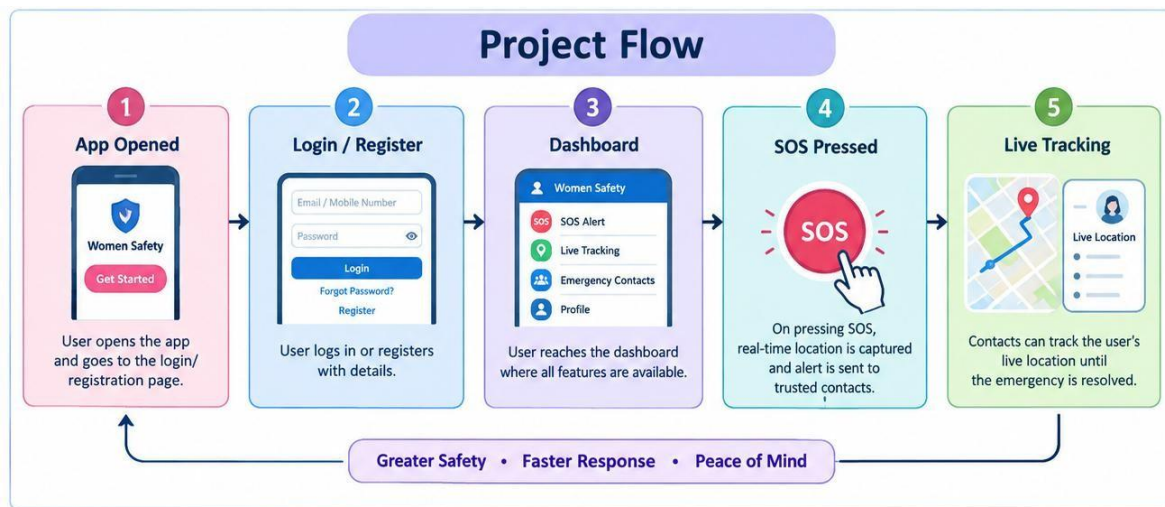


Figure 5.2: Project flow of the Women Safety App (from the project review)

5.2 Use Case Diagram

The use case diagram identifies the actors of the system and the functions they can perform.

There are two actors. The User interacts with all functions of the application. The Emergency Contact does not use the application for login or setup, but receives the SOS alert and watches the live location. The use case “Trigger One-Tap SOS” includes “Capture GPS Location”, and sharing of live location in turn depends on continued location capture.

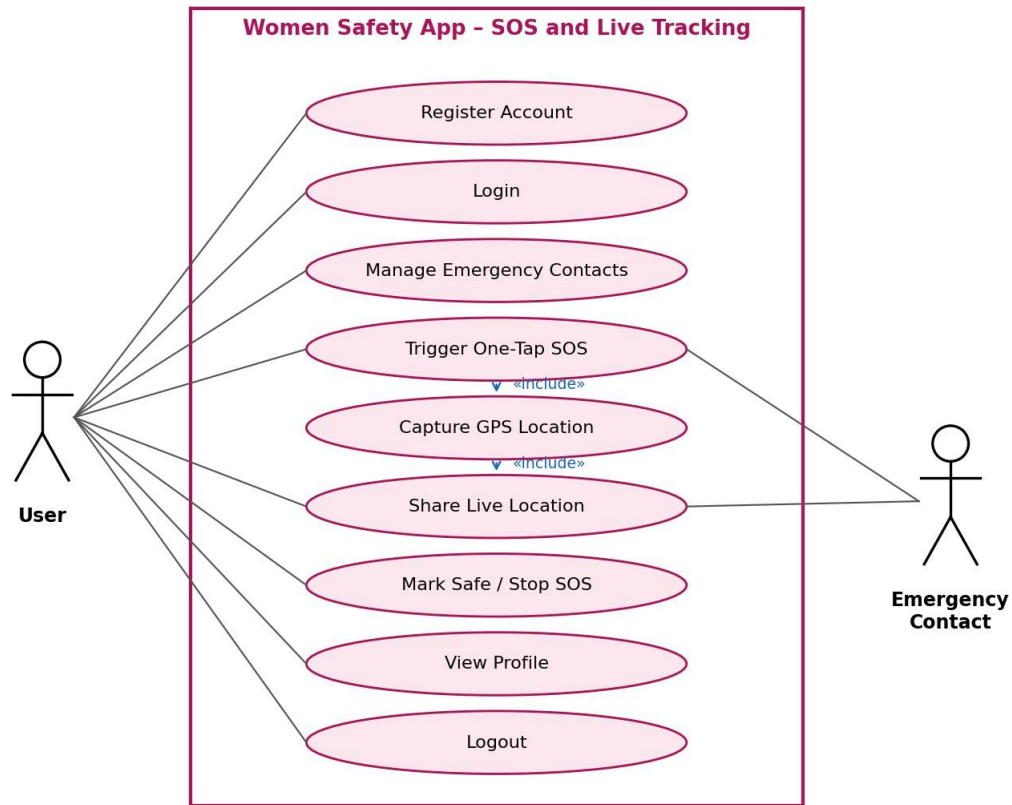


Figure 5.3: Use case diagram

Table 5.1: Description of the main use cases

Use case	Actor	Description
Register Account	User	The user enters name, mobile number, email and password. The system creates the account.
Login	User	The user enters email or phone number and password. On success the dashboard opens.
Manage Emergency Contacts	User	The user adds, edits or removes trusted contacts.
Trigger One-Tap SOS	User	The user presses the SOS button. The system captures the location and sends the alert without confirmation.
Share Live Location	User, Emergency Contact	The system streams location updates; contacts follow them using the live location link.
Mark Safe / Stop SOS	User	The user declares herself safe or cancels the alert, which stops tracking.
View Profile	User	The user views her account details.
Logout	User	The user ends the session.

5.3 Data Flow Diagram

A data flow diagram (DFD) shows how data moves through the system. The context diagram, or Level 0 DFD, shows the whole system as a single process that exchanges data with two external entities: the User and the Emergency Contacts.

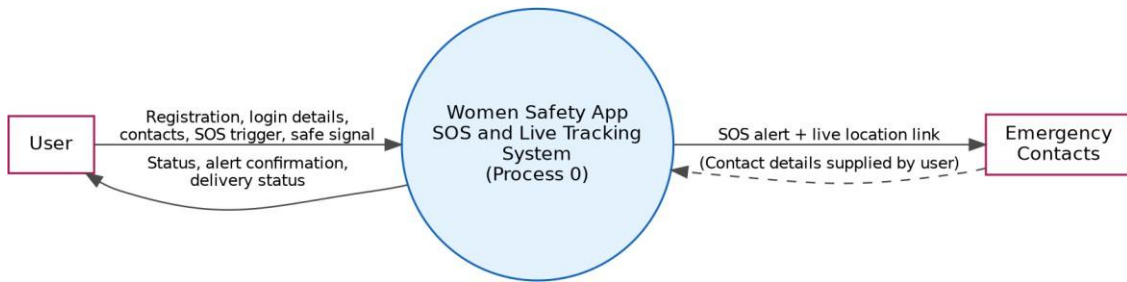


Figure 5.4: Level 0 data flow diagram (context diagram)

The Level 1 DFD divides the system into five processes: Manage Account, Manage Contacts, Process SOS, Live Tracking and Notify Contacts. The data stores are Users, Emergency Contacts, SOS Events, Location Updates and Delivery Logs.

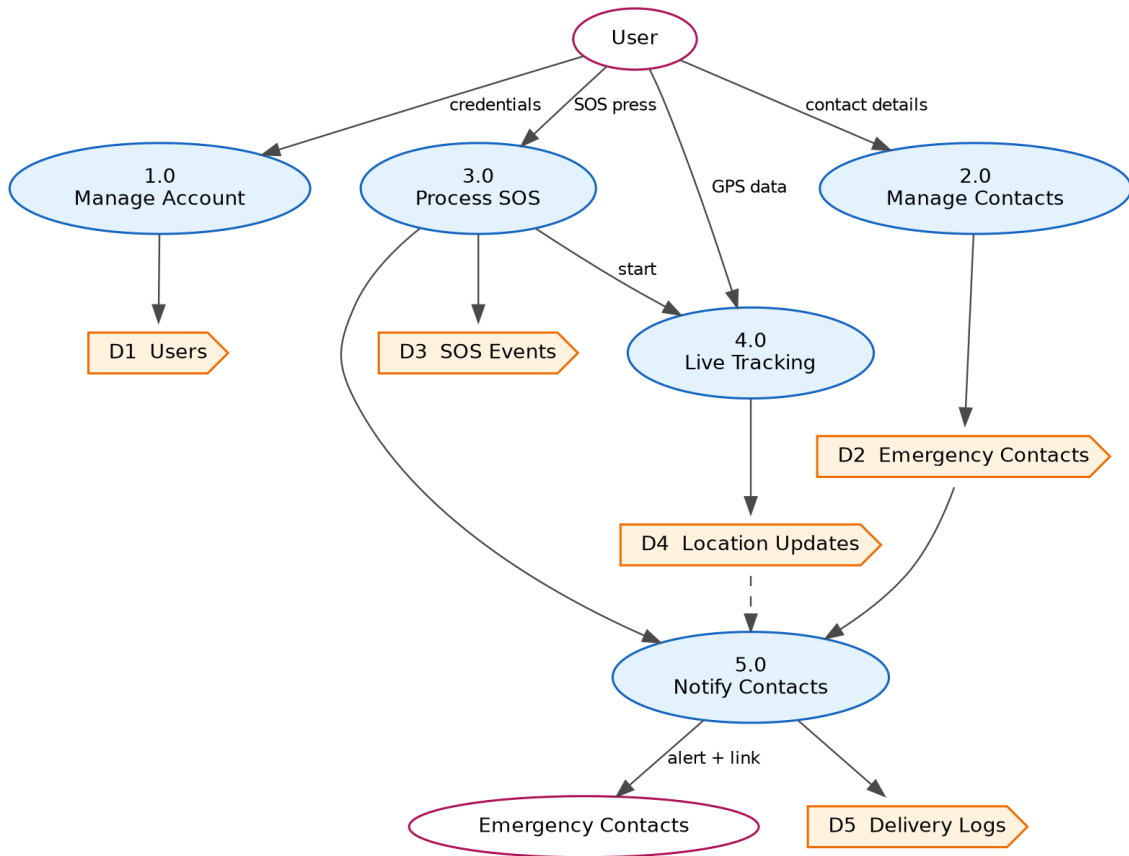


Figure 5.5: Level 1 data flow diagram

5.4 ER Diagram / Database Schema

The database stores five groups of data. The relationships between them are shown in the entity–relationship diagram. A user can have many emergency contacts and can raise many SOS events. Each SOS event records many location updates and generates many delivery log entries, one for each notified contact and channel.

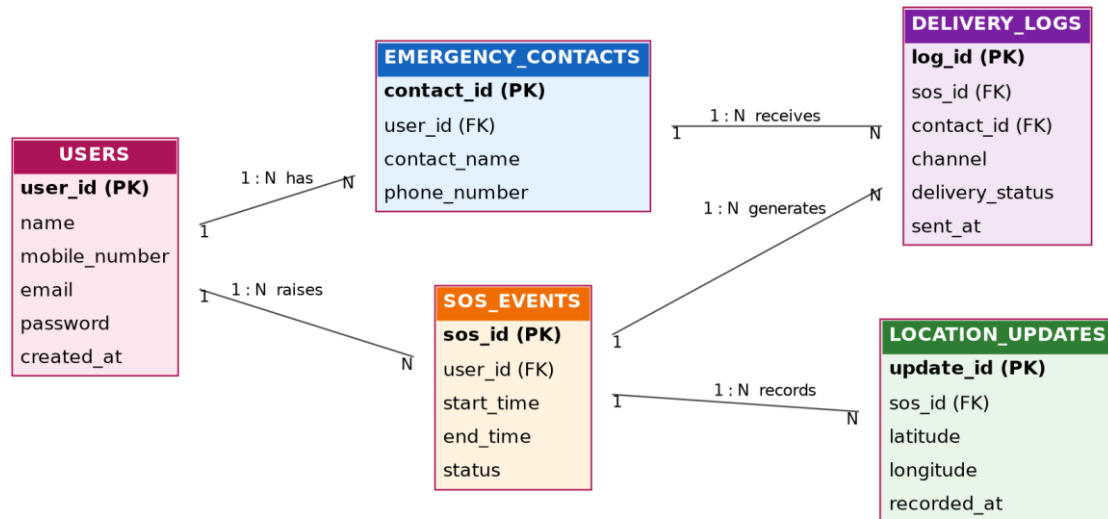


Figure 5.6: Entity–relationship diagram

5.4.1 Database Tables

The tables below are derived from the data groups described in the project: user profile data, emergency contact data, location data and event data. Data types are given as typical choices and may be adjusted to match the database system used.

Table 5.2: Structure of the users table

Field	Type	Description
user_id (PK)	Integer, auto-increment	Unique identifier of the user
name	Text	Name entered at registration
mobile_number	Text	Mobile number of the user
email	Text, unique	Email address used for login
password	Text (protected)	Login credential stored in protected form
created_at	Date-time	Date and time of registration

Table 5.3: Structure of the emergency_contacts table

Field	Type	Description
contact_id (PK)	Integer, auto-increment	Unique identifier of the contact
user_id (FK)	Integer	Owner of the contact; refers to users
contact_name	Text	Name of the trusted guardian
phone_number	Text	Phone number to which the alert is sent

Table 5.4: Structure of the sos_events table

Field	Type	Description
sos_id (PK)	Integer, auto-increment	Unique identifier of the SOS event
user_id (FK)	Integer	User who raised the SOS
start_time	Date-time	Time at which SOS was triggered
end_time	Date-time	Time at which the user marked safe or cancelled
status	Text	Alert status, for example active, safe or cancelled

Table 5.5: Structure of the location_updates table

Field	Type	Description
update_id (PK)	Integer, auto-increment	Unique identifier of the update
sos_id (FK)	Integer	SOS event to which the update belongs
latitude	Decimal	Latitude from the device GPS
longitude	Decimal	Longitude from the device GPS
recorded_at	Date-time	Time at which the position was recorded

Table 5.6: Structure of the delivery_logs table

Field	Type	Description
log_id (PK)	Integer, auto-increment	Unique identifier of the log entry
sos_id (FK)	Integer	SOS event for which the alert was sent
contact_id (FK)	Integer	Contact to whom the alert was sent
channel	Text	Push notification or SMS
delivery_status	Text	Sent, delivered or failed
sent_at	Date-time	Time at which the alert was dispatched

5.5 Activity Diagram

The activity diagram represents the flow of activities from opening the application to the end of an SOS event. A new user first registers and then logs in. From the dashboard the user may manage contacts, view the profile or press SOS. When SOS is pressed, the system captures the GPS position, sends the alert, notifies the contacts and enters continuous live tracking, which continues until the user marks herself safe or cancels the alert.

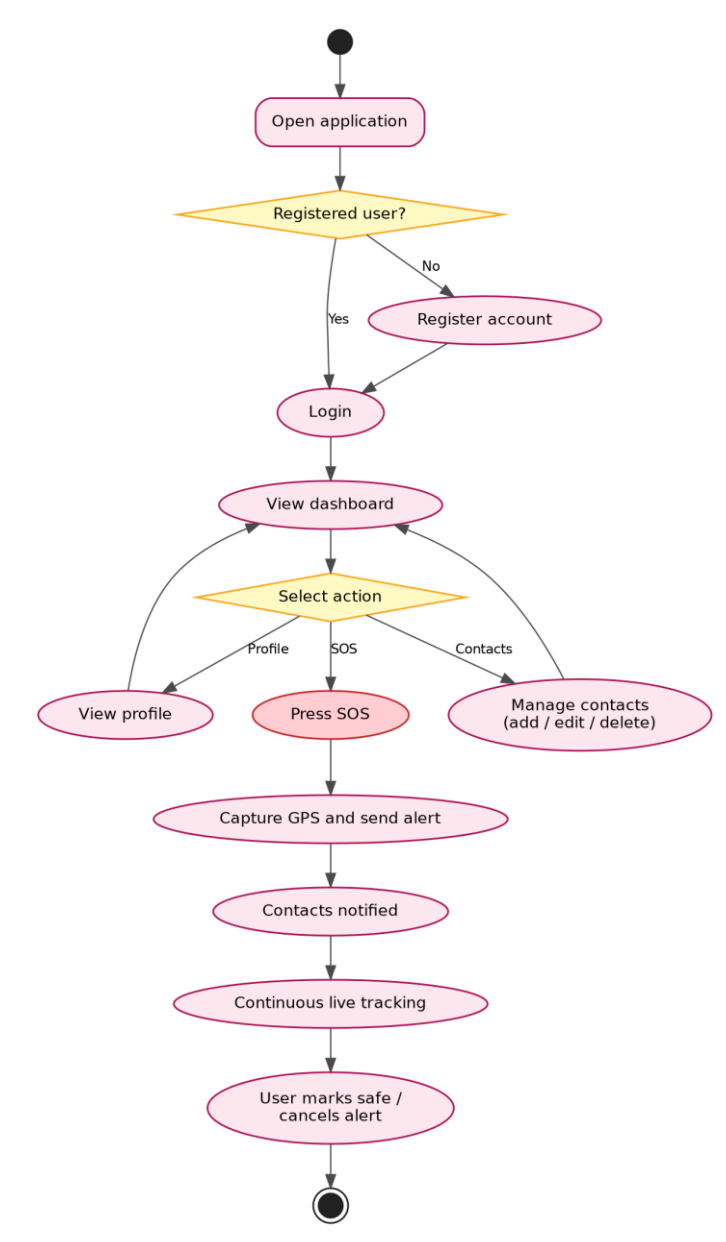


Figure 5.7: Activity diagram

5.6 Sequence Diagram

Sequence diagrams show the order of messages exchanged between objects over time. Four sequence diagrams are given for the most important scenarios: SOS alert, live tracking, login and contact management.

5.6.1 SOS Alert Sequence

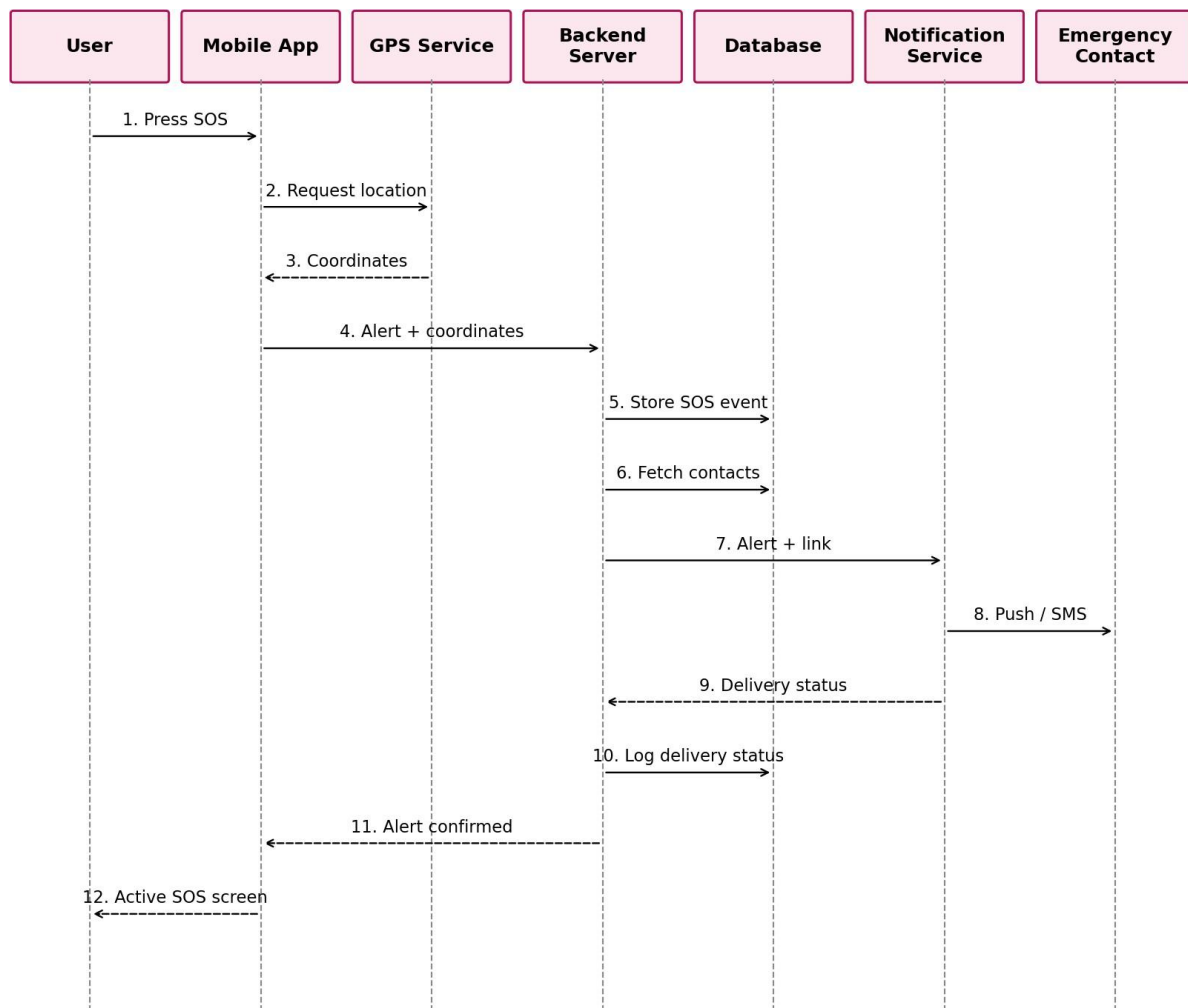


Figure 5.8: Sequence diagram for the SOS alert

5.6.2 Live Tracking Sequence

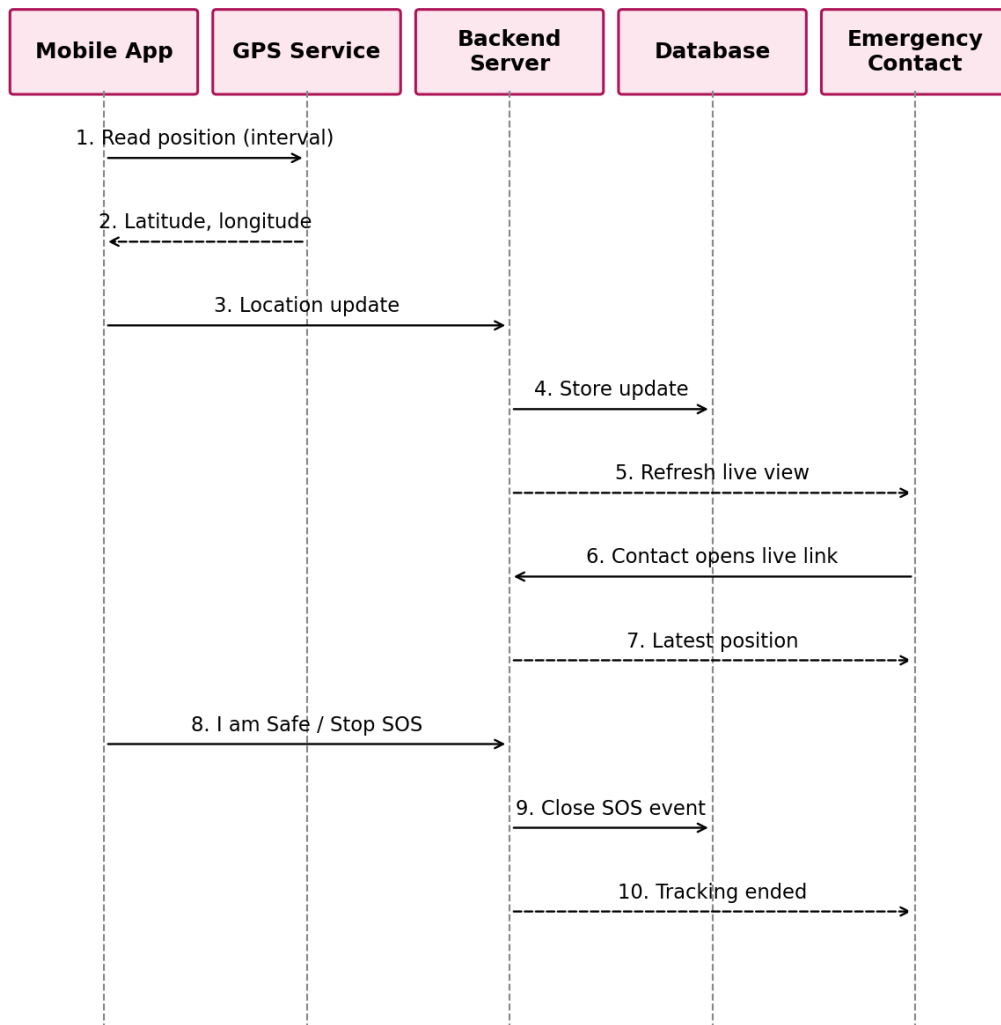


Figure 5.9: Sequence diagram for live tracking

5.6.3 Login Sequence

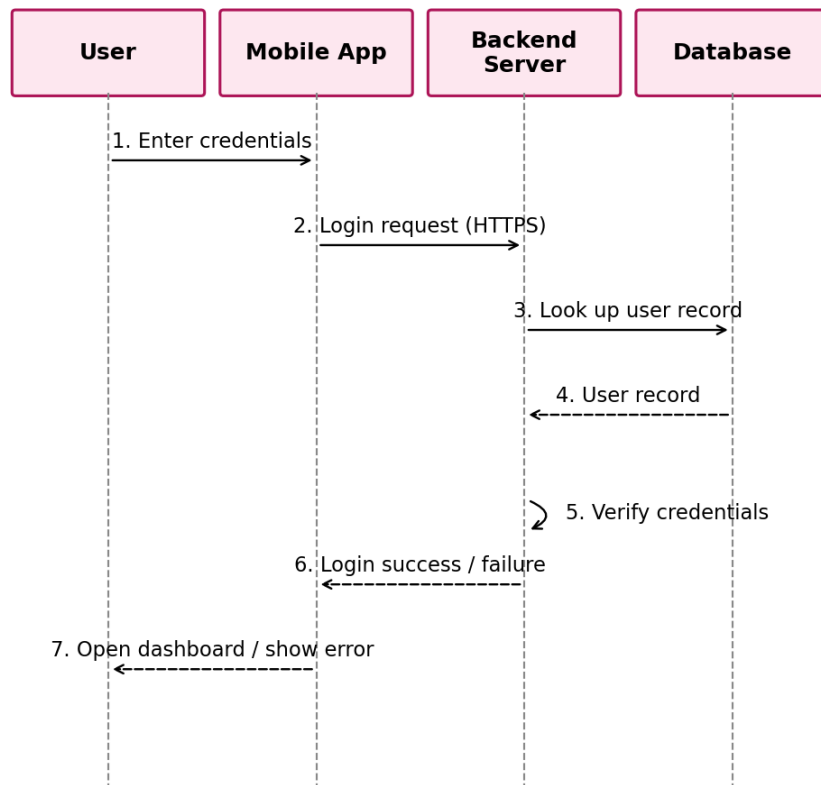


Figure 5.10: Sequence diagram for login

5.6.4 Contact Management Sequence

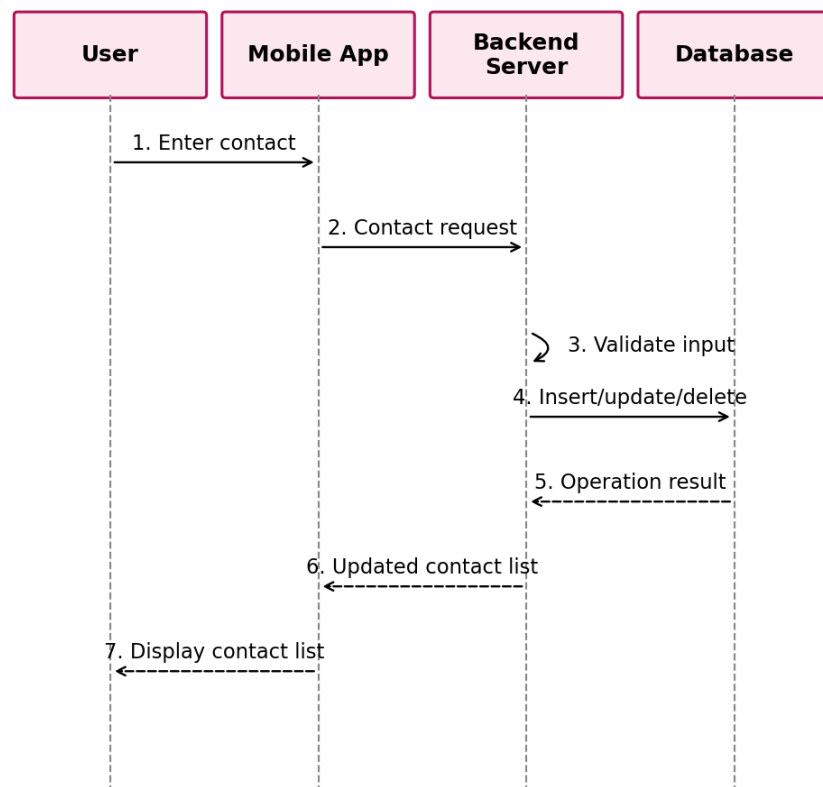


Figure 5.11: Sequence diagram for emergency contact management

5.7 Class / Component Diagram

The class diagram shows the main classes of the system with their attributes and operations.

The User class manages contacts and raises SOS events. Each SOSEvent records location updates and generates notification logs. The EmergencyContact class receives notifications.

These classes correspond directly to the database tables of Section 5.4.

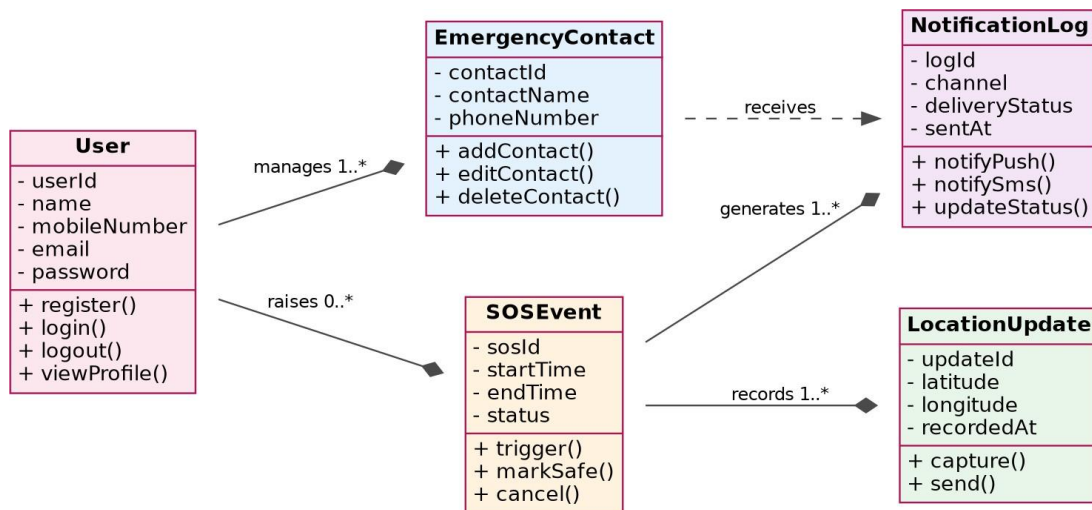


Figure 5.12: Class diagram

5.8 User Interface Design

The user interface was designed to be simple, calm and consistent. A rose-pink and deep-purple colour scheme is used throughout, with rounded buttons and generous spacing. Text is kept short, and each screen has one main purpose. The following screens were implemented, and their screenshots from the working application are shown below.

5.8.1 Welcome Page

The welcome page shows the Women Safety logo with the tagline “Safe Today • Strong Tomorrow”, the message “Welcome to Women Safety App – Your safety is our priority. Stay safe, stay connected.” and a Get Started button that leads to the login or registration page.



Figure 5.13: Welcome page

5.8.2 Login Page

The login page has two fields, Email / Phone Number and Password, a Login button and a Back link. Its simplicity ensures that a returning user can reach the dashboard quickly.

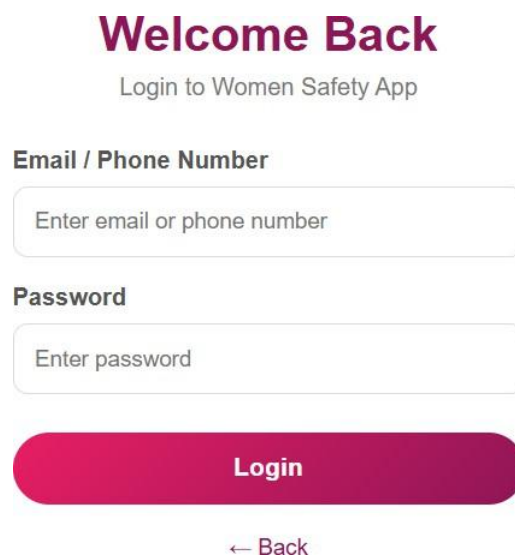


Figure 5.14: Login page

5.8.3 Registration Page

The registration page, titled “Create Account”, asks for four details: Name, Mobile Number, Email and Password, followed by a Register button.

Create Account
Register for Women Safety App

Name
Enter your name

Mobile Number
Enter mobile number

Email
Enter email address

Password
Create password

Register

Figure 5.15: Registration page

5.8.4 Dashboard

The dashboard greets the user with “Welcome to Women Safety App – Your safety is our priority.” A large SOS button is placed at the centre. Below it are three cards: Emergency Contacts (Manage Contacts), Live Tracking (Track Location) and Profile (View Profile).

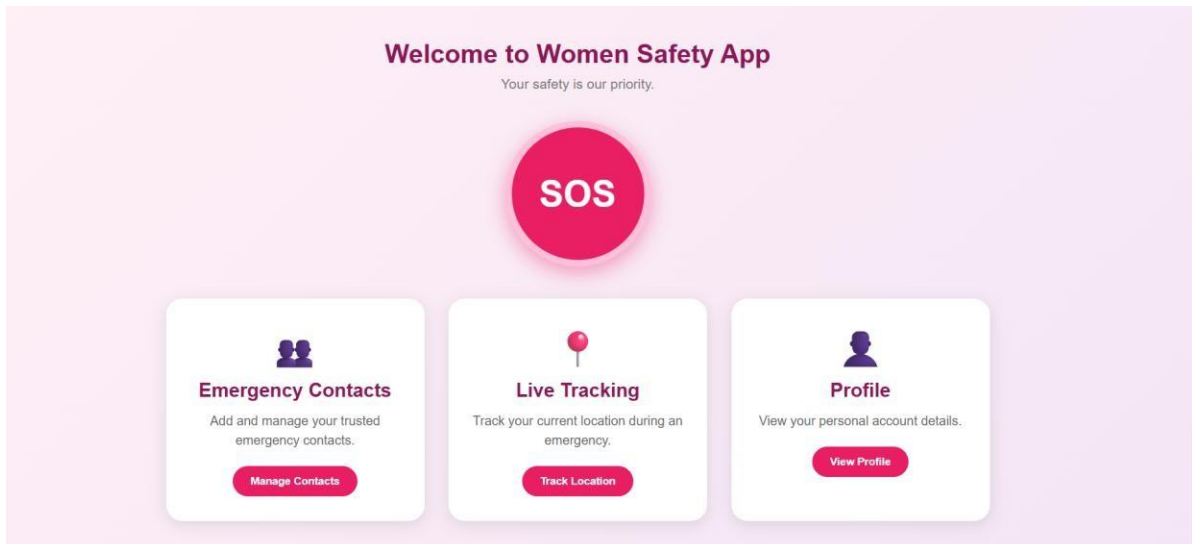


Figure 5.16: Dashboard with SOS button

5.8.5 Other Screens

The Emergency Contacts, Active SOS, Live Tracking and Profile screens complete the interface. Screenshots of these screens were not included in the project review presentation, and they are therefore marked below for insertion.

Emergency Contacts

Add people who should be contacted during an emergency.

Contact Name	Phone Number	Add Contact
--------------	--------------	--------------------

Ananya 9087654321	Delete
harshika 6985478965	Delete
Udaya 8596745896	Delete
nikshitha 6325415625	Delete

← Back to Dashboard

Figure 5.17: Emergency Contacts Page

5.9 Algorithms Used

The application uses simple and predictable procedures, because in a safety application a procedure that is easy to understand is also easier to test and to trust. The three main procedures are given below as numbered steps.

5.9.1 SOS Alert Procedure

1. The user presses the SOS button.
2. The application reads the current GPS coordinates of the device.
3. An alert request is created with the identity of the user, the coordinates and the time.

4. The request is sent to the backend with the highest priority; if the network is weak, it is retried until accepted.
5. The backend stores the SOS event with the status “active”.
6. The backend fetches the emergency contacts of the user.
7. A push notification with the live location link is sent to every contact; if push fails, an SMS is sent.
8. The delivery status of each contact is stored and returned to the user’s device.
9. The live tracking procedure is started.

5.9.2 Live Tracking Procedure

1. While the SOS status is “active”, read the GPS position at the polling interval.
2. If a connection is available, send the position to the backend; otherwise hold it until the connection is stable.
3. The backend stores the update and refreshes the view of the contacts, throttling and deduplicating the updates.
4. When the user marks herself safe or cancels the alert, stop reading the GPS and close the event.

5.9.3 Notification Throttling

The contacts should see the user’s current position without being flooded with messages. The backend therefore compares each new position with the last one that was sent. An update is passed on only if it is new and if enough time has passed since the last one. Repeated identical

updates are dropped. This is the de-duplication and throttling that is described in the project review.

5.10 User Interface Design Guidelines

Table 5.7: Interface guidelines followed in the design

Guideline	Application in the Women Safety App
Prominence of the main action	The SOS button is the largest element on the dashboard and is shown in strong red.
No unnecessary steps	No confirmation dialog or menu between pressing SOS and sending the alert.
Consistency	The same pink and purple colours, rounded buttons and fonts are used on all screens.
Clear labels	Cards are labelled Emergency Contacts, Live Tracking and Profile, with short descriptions.
Short forms	Registration asks only for name, mobile number, email and password; login asks for two fields.
Readable text	Large headings, generous spacing and high contrast between text and background.

CHAPTER – 6

IMPLEMENTATION

This chapter explains how the Women Safety App was implemented. The application was built module by module, and each module was tested on its own before it was integrated with the others. The pseudocode given in this chapter describes the logic of each module in a language-independent form. The actual source code of the project is placed in Appendix 14.2.

6.1 Module-Wise Implementation

The implementation is divided into five modules. Table 6.1 summarises them and shows the sections of this chapter where each module is described.

Table 6.1: Modules of the Women Safety App

Module	Main responsibility	Section
User Account module	Registration, login, logout and authorisation	6.2, 6.11
Profile and Secure Storage module	Display of account details; secure storage of user data	6.3, 6.10
Contact and Notification module	Management of emergency contacts and delivery of alerts	6.4, 6.8
SOS Trigger module	One-tap alert; capture of location	6.5, 6.6
Live Tracking module	Continuous streaming of location until the alert ends	6.7

6.2 User Registration and Login

The registration screen collects the name, mobile number, email address and password of the user. When the Register button is pressed, the application checks that all fields have been filled, and sends a registration request to the backend over the encrypted connection. The backend checks whether the email address is already in use, stores the new user record and returns the result. The login screen takes an email address or phone number and a password. The application sends these to the backend, which looks up the user record, verifies the credentials

and replies with success or failure. On success the dashboard opens; on failure an error message is shown and the user may try again.

```
FUNCTION register(name, mobile, email, password):  
    IF any field is empty THEN show "Please fill all fields"; RETURN  
    SEND registration request to backend  
    IF backend replies "created" THEN open Login screen  
    ELSE show error message from backend  
  
FUNCTION login(identifier, password):  
    IF identifier or password is empty THEN show error; RETURN  
    SEND login request to backend  
    IF backend replies "success" THEN open Dashboard  
    ELSE show "Invalid credentials"
```

6.3 User Profile

The profile page lets the user view her personal account details, namely the name, mobile number and email address that she entered during registration. The page is reached from the Profile card on the dashboard. The details are fetched from the backend for the logged-in user and displayed in read-only form. The profile page also provides the entry point for logging out.

6.4 Emergency Contact Management

Emergency contacts are the people who will receive the alert. The Emergency Contacts screen, reached from the dashboard, lets the user add a new contact by entering a name and a phone number, edit an existing contact and remove a contact that is no longer needed. Each operation is sent to the backend, which updates the `emergency_contacts` table and returns the refreshed list. Because the list is stored on the backend and not only on the phone, it is available to the notification service at the moment of an emergency. The following fields are stored for each contact.

Table 6.2: Emergency contact fields

Field	Type	Purpose
contact_id	Integer (key)	Identifies the contact record

Field	Type	Purpose
user_id	Integer (foreign key)	Links the contact to the owner
contact_name	Text	Name shown to the user and used in delivery status
phone_number	Text	Number to which the alert is delivered

```

FUNCTION addContact(userId, name, phone):
    VALIDATE name is not empty AND phone is a valid number
    SEND "add contact" request to backend
    REFRESH contact list on screen

FUNCTION deleteContact(userId, contactId):
    SEND "delete contact" request to backend
    REFRESH contact list on screen

```

6.5 SOS Trigger Module

The SOS Trigger Module is the entry point of the application and is designed to work with a single tap. When the user presses the SOS button, the module immediately captures the device's current GPS coordinates and packages them with the user's identity into an alert request. This request is sent to the backend server over a lightweight, low-latency connection so that the trigger-to-transmission time stays minimal even under stress. The module is intentionally kept free of confirmation dialogs or multi-step menus, since every extra tap increases response time during a genuine emergency. Once the alert is sent, the module hands off control to the live tracking module so that location sharing begins automatically without any further action from the user.

```

FUNCTION onSOSPressed():
    position = getCurrentLocation()           // see 6.6
    request = { userId, position.lat, position.lng, currentTime }
    sendWithPriority(request)                 // initial alert has highest
priority
    startLiveTracking(userId)                 // hand-off, see 6.7
    showActiveSOSScreen()

```

6.6 GPS / Location Capture

Location capture is performed using the location services of the device. The application requests permission to access the device location when it is first used. When the SOS button is pressed, the application reads the most recent available position and, if necessary, requests a fresh one. The result consists of a latitude and a longitude. These coordinates, together with a timestamp, form the core of both the initial alert and the later location updates. The application relies on the GPS receiver, which is why a GPS-enabled device is listed as a requirement in Chapter 2.

If the user has not granted location permission or has switched off location services, the application cannot capture coordinates. In that case the problem must be made visible to the user, as discussed in Section 6.13 and in Chapter 10.

6.7 Live Location Tracking

The Live Location Tracking Module takes over once an SOS alert is active. It continuously reads the device's GPS position at short, regular intervals and streams these updates to the backend server, which in turn keeps the trusted contacts' view of the user's location current. This continuous stream, rather than a single snapshot, allows guardians to follow the actual movement of the user, for example tracing a route home, instead of seeing only where the emergency began.

The module is optimised to balance location accuracy with battery and data usage, and it keeps running in the background even if the user locks the screen or switches to another app. Tracking automatically stops once the user marks herself safe or the alert is manually cancelled.

```
FUNCTION startLiveTracking(userId):  
    WHILE sosStatus == ACTIVE:
```

```

position = getCurrentLocation()
IF connection is usable THEN
    sendLocationUpdate(userId, position, currentTime)
ELSE
    hold update until connection stabilises
    WAIT for the tuned polling interval
stopLocationUpdates()

```

6.8 Emergency Alert Module

The Contact and Notification Module is responsible for delivering alerts reliably. When an SOS is triggered, the backend retrieves the current contact list of the user and dispatches alert notifications, including the live location link, through a push-notification service. Where push delivery is not possible, SMS is used as a fallback. The delivery status of each contact is recorded in the `delivery_logs` table so that the user's device can confirm that the guardians have actually received the alert.

The module also throttles and deduplicates the location updates sent to contacts. Without this control, a guardian could receive a notification for every single coordinate, which would be overwhelming. With it, notifications remain timely but are not repeated unnecessarily.

```

FUNCTION notifyContacts(sosId, userId):
    contacts = fetchContacts(userId)
    FOR EACH contact IN contacts:
        result = sendPush(contact, liveLocationLink(sosId))
        IF result is not delivered THEN
            result = sendSMS(contact, liveLocationLink(sosId))
        saveDeliveryLog(sosId, contact, result)
    RETURN delivery summary to the user device

```

6.9 Backend API Integration

The mobile application communicates with the backend through a set of requests. Each request carries the user's identity so that the backend knows whose data is being accessed. The main groups of requests are listed in Table 6.3.

Table 6.3: Backend request groups used by the application

Group	Purpose	Sent by
Authentication	Register a new user; verify login	Registration and login screens
Profile	Return the account details of the user	Profile screen
Contacts	Add, edit, delete and list emergency contacts	Contacts screen
SOS alert	Create an SOS event and start notification	SOS Trigger module
Location update	Store a new coordinate pair for an active SOS	Live Tracking module
Stop SOS	Mark the event as safe or cancelled and stop tracking	I'm Safe / Cancel control

6.10 Database Integration

The backend is the only component that reads from and writes to the database. Registration creates a row in the users table; contact operations change rows in emergency_contacts; each SOS creates a row in sos_events; each tracking update adds a row in location_updates; and each notification adds a row in delivery_logs. Keys and relationships keep these records consistent, as shown in the ER diagram of Section 5.4. Since location data is sensitive, location history is stored securely and for a limited time only.

6.11 Authentication and Authorization

Authentication confirms who the user is, and authorization decides what the user may do. Every function other than registration and login requires a logged-in user. The backend uses the identity of the user to limit each request to that user's own data: a user can see and change only her own profile and contacts, and can start, update and stop only her own SOS events. Emergency contacts are not given general access to the application; they receive only a live location link for the specific SOS event. All requests travel over an encrypted connection, and user data is stored securely.

6.12 Frontend Logic

The frontend logic controls what the user sees and what happens after each action. The welcome page leads to login or registration, and after login the dashboard is displayed. Navigation from the dashboard is intentionally shallow, with one tap leading to each feature. When SOS is pressed, the active SOS screen is shown immediately and the user is not required to do anything further. The application keeps track of the state of the current alert, either inactive or active. This state decides whether the location polling loop continues. When the user marks herself safe or cancels, the state changes to inactive and tracking stops.

Table 6.4: States of an SOS event

State	Meaning	Next state
Inactive	No emergency; normal use of the app	Active (on SOS press)
Active	Alert sent; live tracking running	Safe or Cancelled
Safe	User marked herself safe; tracking stopped	Inactive
Cancelled	Alert cancelled manually; tracking stopped	Inactive

6.13 Error Handling

The application is designed to fail in a visible and safe manner. The principal error situations and the way they are handled are described in Table 6.5.

Table 6.5: Error situations and handling

Situation	Handling	Module
Empty or invalid input	The form is not submitted; a message identifies the missing field	Frontend
Wrong login details	An “invalid credentials” message is shown; the user may retry	Authentication
Location permission denied or GPS off	The user is told that location is required for SOS	SOS Trigger
Weak or no network	The initial alert is prioritised and sent as soon as a connection is usable; location streaming resumes later	SOS Trigger / Tracking

Situation	Handling	Module
Push delivery failure	SMS is used as a fallback and the result is logged	Notification
Backend or database error	An error response is returned; the app shows a message and does not crash	Backend

6.14 Input Validation Rules

Validation prevents wrong or incomplete data from entering the system. The checks are applied by the frontend, so that the user gets quick feedback, and again by the backend, because the backend cannot assume that every request came from a correct frontend.

Table 6.6: Validation rules

Field	Rule	Message to user
Name	Must not be empty	Please enter your name
Mobile number	Must not be empty; must contain a valid number of digits	Please enter a valid mobile number
Email	Must not be empty; must be in a valid email format; must be unique	Please enter a valid email / email already registered
Password	Must not be empty; a minimum length may be required	Please create a password
Login identifier	Email or phone number must not be empty	Please enter email or phone number
Contact name	Must not be empty	Please enter the contact name
Contact phone	Must not be empty; must be a valid number	Please enter a valid phone number

6.15 Life Cycle of a Request

Every request from the application to the backend passes through the same steps, which is why the behaviour of the system is easy to follow. As an example, consider a request to add an emergency contact.

1. The user enters the name and number and presses the save button.
2. The frontend validates the entries and creates the request, with the identity of the user.

3. The request is sent to the backend over the encrypted connection.
4. The backend identifies the user, validates the data again and writes a record in the emergency_contacts table.
5. The backend sends back a response which states whether the operation succeeded.
6. The frontend updates the list shown on the screen, or shows an error message.

The SOS request follows the same pattern, except that the backend, after storing the event, hands the work to the notification service and returns to the application without waiting for every notification to be finished, so that the user's screen is updated at once.

CHAPTER – 7

FEATURES

7.1 Main Features

The features of the Women Safety App are intentionally few, because every feature in a safety application competes for the attention of a user who may be frightened. The main features are listed below, and each is described from two points of view: how the user experiences it, and how it works technically.

- User registration and login
- Management of emergency contacts
- One-tap SOS with no confirmation step
- Real-time capture of the current GPS location
- Continuous live location tracking with a shareable live location link
- Emergency alert to all contacts by push notification, with SMS fallback
- Display of the active SOS state and delivery confirmation
- “I’m Safe” / Stop SOS to end the alert
- Profile viewing and logout
- Secure storage and encrypted transmission of data

Table 7.1: Features and the objectives they satisfy

Feature	Objective satisfied
One-tap SOS	Single-tap mechanism that instantly alerts trusted contacts
Live tracking	Continuous GPS tracking that guardians can follow until the situation is resolved
Dashboard interface	Simple interface that works with minimal interaction under stress

Feature	Objective satisfied
Emergency contact management	Secure adding, editing and removing of trusted guardians
Priority alert delivery	Fast and dependable delivery under limited connectivity
Secure storage and encryption	Protection of user data and location history

7.2 User Registration

User perspective: A new user taps Get Started on the welcome page and then selects Register. She types her name, mobile number, email address and a password and presses Register. After her account is created she can log in.

Technical working: The frontend checks the fields and sends a registration request. The backend verifies that the email address is not already in use, stores the user record in the users table and confirms creation.

7.3 Login

User perspective: The returning user enters her email or phone number and password and presses Login. If the details are correct the dashboard opens at once. A Back link returns her to the welcome page.

Technical working: The credentials are sent to the backend over an encrypted connection. The backend looks up the user record, verifies the credentials and returns the result. Only after a successful login is the dashboard opened.

7.4 Emergency Contacts

User perspective: From the Emergency Contacts card the user can add several trusted contacts such as parents, siblings or friends, change their details at any time and remove them. She should add contacts before she needs the application, because the SOS alert is sent to the contacts already stored.

Technical working: Each operation is a request to the backend, which writes to the emergency_contacts table. At the moment of an SOS, the notification service reads this table to find the recipients.

7.5 One-Tap SOS

User perspective: The SOS button is the largest element on the dashboard. A single press raises the alarm. There is no confirmation dialog and no menu, since every extra step would cost time. The user need not speak, type or explain anything.

Technical working: The SOS Trigger Module captures the current GPS coordinates, prepares an alert request with the user's identity and sends it to the backend with the highest priority. It then starts the live tracking module.

7.6 Current Location

User perspective: The user does not have to find or type her location. It is captured automatically at the instant of the SOS.

Technical working: The application reads the latitude and longitude from the device's location services. The coordinates are sent with the alert and are stored with a timestamp. Location permission must be granted for this feature to work.

7.7 Live Tracking

User perspective: After SOS, sharing continues without any action by the user, even if she locks the screen or opens another application. The Live Tracking card on the dashboard also lets her view her current location during an emergency.

Technical working: The Live Tracking Module reads the GPS position at short, regular intervals and sends each reading to the backend, which updates the view seen by the contacts. The polling interval is tuned to balance accuracy with battery and data use. Updates are throttled and deduplicated before they are sent to contacts.

7.8 Emergency Alert

User perspective: The user's trusted contacts receive an alert message with a live location link. Through this link they can follow the user's movement and decide whether to call her, go to her location or ask for help.

Technical working: The backend fetches the contact list and sends push notifications that contain the live location link. If push delivery is not possible, an SMS is sent instead. The delivery status of each contact is recorded and sent back to the user's device as confirmation.

7.9 Active SOS

User perspective: After the alert has been sent, the application shows that an SOS is active, so the user knows that help has been alerted and that her location is being shared.

Technical working: The application keeps the SOS state as active. This state controls the tracking loop and stays active until the alert is ended by the user.

7.10 I'm Safe / Stop SOS

User perspective: When the emergency is over, the user marks herself safe or cancels the alert. Sharing of her location then stops. This keeps her privacy under her own control and tells her contacts that the situation is resolved.

Technical working: The application changes the SOS state, stops reading the GPS and informs the backend, which records the end time and the final status of the event in the `sos_events` table.

7.11 Profile Management

User perspective: The Profile card shows the user's personal account details, so that she can check that the information registered in the application is correct.

Technical working: The backend returns the profile record of the logged-in user, which the application displays in read-only form.

7.12 Logout

User perspective: The user can log out when she has finished, so that nobody else who uses her phone can open her account.

Technical working: Logout ends the session on the device and returns the user to the login or welcome page.

7.13 User Perspective

From the point of view of the user, the application can be described in five steps: open the app, log in, reach the dashboard, press SOS if in danger, and let the app share the location until she is safe. Everything else, such as contact management and profile, is done earlier, in calm

conditions. The design deliberately separates preparation from emergency use, so that the emergency path is as short as possible.

7.14 Technical Working

Technically, the application works as a chain of events. The tap on SOS triggers the capture of coordinates. An alert request is sent to the backend, which stores the event and passes the work to the notification service. The notification service reaches the guardians, and the delivery status returns to the user's device. Meanwhile, the tracking module continues to stream coordinates, and the backend keeps the guardians' view fresh. The process is closed when the user ends the alert. Table 7.2 lists the events and the module responsible for each.

Table 7.2: Chain of events during an emergency

Step	Event	Responsible module
1	SOS button pressed	Frontend / SOS Trigger
2	Current GPS coordinates captured	SOS Trigger
3	Alert request sent with priority	SOS Trigger
4	SOS event stored	Backend / Database
5	Contacts fetched	Contact and Notification
6	Push alert with live link sent; SMS if push fails	Contact and Notification
7	Delivery status recorded and returned to the user	Contact and Notification
8	Location streamed at regular intervals	Live Tracking
9	Guardians follow the live location	Notification / Live link
10	User marks safe or cancels; tracking stops	Frontend / Backend

7.15 An Illustrative Scenario

The following scenario is given only to show how the features work together. It is not a record of a real event.

A student is returning home by a lonely road after an evening class. Before the day began she had registered in the application and added her mother, her brother and a close friend as emergency contacts. On the road she notices that she is being followed and feels unsafe. She takes out her phone, which is already logged in, and presses the large SOS button once.

At that moment the application reads her GPS position and sends an alert to the backend. Within a few seconds her mother, brother and friend each receive a notification with a link to her live location. The screen of the student now shows that the SOS is active, and she puts the phone in her pocket. As she continues to walk, her position is sent at regular intervals, and her family can see on the map that she is moving towards a main road. Her brother decides to go and meet her, and her friend calls the nearest relative who lives close to that road.

When she reaches a safe place, the student marks herself safe in the application. The sharing of her location stops, and her family knows that the situation has ended. The scenario shows the main strengths of the design: a single action, no need to explain anything, simultaneous alert to several people, continuous tracking and control by the user over when the sharing ends.

CHAPTER – 8

TESTING

8.1 Testing Introduction

Testing is the process of checking whether the software behaves as expected and finding faults before the software is used. For a safety application, testing has a special importance, because a failure is not just an inconvenience: it could mean that an alert is not delivered at a moment when it is needed. The testing of the Women Safety App was therefore planned around the question: does an SOS reach the trusted contacts quickly and reliably, in the conditions in which real emergencies happen?

The experimental setup used the following arrangement, as described in the project review.

Table 8.1: Experimental setup

Item	Description
Devices	Mid-range Android smartphones with GPS enabled, used to simulate typical end-user hardware
Network conditions	Strong Wi-Fi, moderate 4G and weak or intermittent mobile data
Test scenarios	Repeated SOS triggers from stationary and moving positions to check alert and tracking accuracy
Contacts setup	Each test account was configured with several trusted contacts to verify simultaneous delivery
Metrics captured	Trigger-to-notification time, location update frequency and alert delivery success rate
Test data	Simulated SOS scenarios and varied network conditions

The testing activities were arranged in layers. Unit testing checked the individual modules; integration testing checked how the modules work together; and system-level tests, such as location testing, SOS testing and performance testing, checked the behaviour of the application in realistic scenarios.

8.2 Unit Testing

In unit testing each module was tested separately, before it was connected to the others. The SOS Trigger Module was tested to confirm that it captured the coordinates and created a correct alert request. The Live Tracking Module was tested to confirm that it read the position repeatedly at its polling interval and stopped when the alert ended. The Contact and Notification Module was tested to confirm that it retrieved the right contacts and recorded the delivery status. The account functions, registration and login, were tested with valid and invalid input.

8.3 Integration Testing

Integration testing checked that the modules cooperate correctly. The most important integration is the hand-off from the SOS Trigger Module to the Live Tracking Module, which must start automatically without any extra action from the user. Another is the chain from the alert request through the backend and the database to the notification service. The tests confirmed that the contacts consistently received both the initial alert and a working live-location link, and that the live tracking module kept the contacts' view of the user's position updated at short, regular intervals.

8.4 Frontend Testing

Frontend testing examined the screens of the application: welcome, registration, login, dashboard, contacts and profile. The tests checked that the fields accept input, that the buttons lead to the correct screens, that error messages appear for missing or wrong input, and that the SOS button is prominent and works with a single tap. The behaviour of the application with

the screen locked, or when another application is open, was also checked for the tracking function.

8.5 Backend Testing

Backend testing verified that the server processes each kind of request correctly. Registration requests with new and with existing email addresses were sent, as were login requests with correct and incorrect passwords. SOS requests were sent to confirm that an event is stored and that notifications are triggered. Requests with missing data were sent to confirm that the backend returns a clear error and does not stop.

8.6 API Testing

The requests that the mobile application sends to the backend were tested independently of the mobile application, using an API testing tool. Each request type listed in Table 6.3 was sent with valid and invalid data, and the responses were checked for the correct content and status. This made it possible to find backend faults before the integration with the mobile screens, which reduced the time spent on debugging.

8.7 Database Testing

Database testing confirmed that the data is stored correctly and that the relationships between tables are maintained. Records inserted by registration, contact operations, SOS events, location updates and delivery logs were inspected directly in the database. Queries were run to verify that the contacts of a user can be retrieved, that all location updates of an event can be listed in time order, and that deleting a contact does not damage other records.

8.8 Location Testing

Location testing checked whether the coordinates sent by the application correspond to the actual position of the device. Tests were done from stationary positions and from moving positions. The results showed that the live location updates stayed closely aligned with the device's actual GPS position throughout testing. The tests also checked the behaviour when the location permission was denied and when location services were switched off.

8.9 SOS Testing

SOS testing is the central test of the project. The SOS button was pressed repeatedly, with different numbers of contacts, from stationary and moving positions and under three network conditions: strong Wi-Fi, moderate 4G, and weak or intermittent mobile data. Testing confirmed that the SOS Trigger Module reliably captured the device's location and dispatched an alert within a couple of seconds of the button press under normal network conditions. Even under weak or intermittent connectivity, the system prioritised the initial SOS alert so that contacts were notified before location streaming resumed once the connection stabilised.

8.10 Test Cases

The test cases are listed in Tables 8.2 to 8.4. The status "Passed" is entered only for the behaviours that were confirmed in the project review results. The remaining entries are marked for the team to record after the tests are executed on the final build.

Table 8.2: Test cases for registration and login

ID	Scenario	Input	Expected result	Status
TC01	Register with valid details	Name, mobile, email, password	Account created	[Record]
TC02	Register with empty field	Name left blank	Error message; no account created	[Record]

ID	Scenario	Input	Expected result	Status
TC03	Register with an email already used	Existing email	Registration rejected with message	[Record]
TC04	Login with correct details	Valid email and password	Dashboard opens	[Record]
TC05	Login with wrong password	Valid email, wrong password	Error message; no access	[Record]
TC06	Login with empty fields	Nothing entered	Error message	[Record]
TC07	Logout	Press logout	Session ends; login page shown	[Record]

Table 8.3: Test cases for emergency contacts

ID	Scenario	Input	Expected result	Status
TC08	Add a contact	Name and phone number	Contact appears in list	[Record]
TC09	Add a contact with empty number	Name only	Error; contact not saved	[Record]
TC10	Edit a contact	Change phone number	Updated number saved and shown	[Record]
TC11	Delete a contact	Select delete	Contact removed from list and database	[Record]
TC12	Multiple contacts configured	Several contacts for one account	All contacts stored and listed	[Record]

Table 8.4: Test cases for SOS, tracking and alerts

ID	Scenario	Input	Expected result	Status
TC13	SOS under normal network	Press SOS on strong Wi-Fi or 4G	Location captured and alert dispatched within a couple of seconds	Passed
TC14	Contacts receive alert and link	SOS with configured contacts	Contacts receive the alert and a working live-location link	Passed
TC15	Live tracking while moving	SOS while moving	Contacts' view updated at short regular intervals	Passed
TC16	SOS under weak connectivity	Press SOS on weak or intermittent data	Initial alert is sent first; streaming resumes when the connection stabilises	Passed
TC17	Several contacts notified at once	SOS with multiple contacts	Delivery stays consistent for all contacts	Passed
TC18	Push delivery fails	Contact unreachable by push	SMS fallback is used; result logged	[Record]

ID	Scenario	Input	Expected result	Status
TC19	Mark safe	Press I'm Safe	Tracking stops; event closed	[Record]
TC20	Cancel alert	Cancel active SOS	Tracking stops; status cancelled	[Record]
TC21	Background tracking	Lock screen after SOS	Updates continue in background	[Record]
TC22	Location permission denied	SOS without permission	User is told location is required	[Record]

8.11 Test Results

Testing confirmed that the single-tap trigger, continuous tracking and contact-notification pipeline work together as intended. The principal results are summarised in Table 8.5.

Table 8.5: Summary of test results (as reported in the project review)

Area	Result
SOS trigger	Reliably captured the device location and dispatched an alert within a couple of seconds of the button press under normal network conditions
Contact notification	Trusted contacts consistently received both the initial alert and a working live-location link
Live tracking	Contacts' view of the user's position was updated at short, regular intervals in all test scenarios
Weak connectivity	The initial SOS alert was prioritised, so contacts were notified before location streaming resumed
Multiple contacts	Alert delivery remained consistent when several contacts were notified simultaneously

8.12 Error Handling

The tests also checked the error handling described in Section 6.13. Wrong or incomplete input produced messages and not crashes; permission and connectivity problems were reported to the user; and a failed push delivery caused the SMS fallback to be used. These tests are important because the way an application behaves in unusual conditions is as important as its normal behaviour.

8.13 Performance Testing

Performance testing measured how fast and how reliably the application reacts. The three metrics captured were trigger-to-notification time, location update frequency and alert delivery success rate. Table 8.6 gives the observations that were reported at the project review. Because no numerical figures were recorded in the review, the column for measured values is left for the team to fill in with the values obtained from the final build, such as the average trigger-to-notification time in seconds.

Table 8.6: Performance evaluation

Metric	Observation	Measured value	Conditions
Response time	SOS alerts reached contacts within a few seconds on average under normal connectivity	[Enter value]	Wi-Fi / 4G
Tracking accuracy	Live updates stayed closely aligned with the actual GPS position	[Enter value]	Stationary and moving
Reliability under load	Delivery remained consistent when several contacts were notified simultaneously	[Enter value]	Multiple contacts
Low-connectivity performance	Initial alert prioritised over tracking when bandwidth was limited	[Enter value]	Weak data
Battery and data usage	Polling intervals tuned to balance freshness against battery and data consumption	[Enter value]	Active tracking

8.14 Test Environment and Test Data

The tests used simulated data and not real emergencies. Accounts were created for the test purposes, each with several trusted contacts who were members of the team or their acquaintances who had agreed to take part. Simulated SOS events were raised from stationary and moving positions. The network conditions were changed deliberately so that the behaviour under strong, moderate and weak connectivity could be compared.

Table 8.7: Test data used

Data	Description
User accounts	Test accounts with name, mobile number, email and password
Emergency contacts	Several contacts for each test account
SOS events	Simulated SOS triggers from stationary and moving positions
Network conditions	Strong Wi-Fi, moderate 4G, weak or intermittent mobile data
Invalid inputs	Empty fields, wrong passwords, repeated email, missing contact number

8.15 Defect Log

Faults found during testing should be recorded, with the cause and the action taken, so that nothing is forgotten. Table 8.8 gives the format of the defect log for the team to complete from its own test records.

Table 8.8: Defect log format

No.	Defect found	Module	Action taken	Status
1	[Enter defect]	[Module]	[Fix applied]	[Closed]
2	[Enter defect]	[Module]	[Fix applied]	[Closed]
3	[Enter defect]	[Module]	[Fix applied]	[Closed]

8.16 Testing Summary

The testing of the Women Safety App was designed around the real conditions of use. Each module was tested alone, the modules were then tested together, and the whole system was

tested under strong Wi-Fi, moderate 4G and weak or intermittent data. The results reported in the project review show that the application meets its core aim: the SOS is dispatched within a couple of seconds under normal conditions, contacts receive both the alert and a working live-location link, tracking updates continue at short intervals, and the initial alert has priority when the network is weak.

The testing also shows what remains to be done. The numerical values of response time and delivery success rate should be recorded in the final build, the entries marked for recording in the test tables should be completed, and the application should be tested on a wider range of devices and in more real-world movement conditions. These points are included among the limitations in Section 10.8.

CHAPTER – 9

DEPLOYMENT

9.1 Deployment Introduction

Deployment is the process of making the application available for use. In the case of the Women Safety App, deployment has two parts. The mobile application must be installed on the smartphone of the user, and the backend server with its database must be running and reachable from that smartphone. This chapter describes the steps in a general form, from preparing the development environment to running the application. The commands and configuration values depend on the tools used, and so they are given in the generic form and the team should enter the exact values in Appendix 14.3.

9.2 Development Environment

The development environment is set up on a computer with the tools listed in Chapter 2. The mobile application development tools and an emulator are installed on it, along with the backend runtime and the database software. A GPS-enabled Android smartphone, set to allow installation of test builds, is connected for real-device testing. All of these components must be able to reach each other, either on the same computer or through the same local network.

9.3 Frontend Setup

1. Install the mobile application development environment and its required components.
2. Open the project folder of the mobile application in the environment.
3. Allow the environment to download the libraries that the project depends on.
4. Configure the address of the backend server in the application settings.
5. Check that the location permission is declared, so that the device asks the user for it.

6. Connect an Android phone or start the emulator.
7. Build and run the application.

9.4 Backend Setup

1. Install the server-side runtime required by the backend.
2. Open the backend project folder and install its dependencies.
3. Provide the configuration values described in Section 9.6.
4. Start the backend server and confirm that it is listening for requests.
5. Send a test request with an API testing tool to confirm that it responds.

9.5 Database Setup

1. Install the database software, or create a cloud database instance.
2. Create a database for the application.
3. Create the tables `users`, `emergency_contacts`, `sos_events`, `location_updates` and `delivery_logs` as defined in Section 5.4.
4. Create a database user with limited permissions for the backend.
5. Enter the connection details in the backend configuration.

9.6 Environment Configuration

Environment configuration keeps settings such as addresses, secrets and keys out of the program code. This is useful for security, because secrets are not written in source files that might be shared, and for flexibility, because the same code can run on a development machine

and on a server with different settings. The settings needed by the Women Safety App are listed in Table 9.1.

Table 9.1: Configuration settings required by the application

Setting	Purpose	Where used
Server address and port	Where the backend listens for requests	Backend
Database connection details	Host, database name, user and password	Backend
Location / maps API key	Authorises use of the mapping service	Frontend / live link
Push-notification credentials	Authorises the backend to send push messages	Backend
SMS service credentials	Authorises sending of fallback SMS	Backend
Backend address in the app	Where the app sends its requests	Frontend

9.7 Running the Application

To run the system, first start the database, then the backend, and finally the mobile application. Once the application is running on the phone, the following check confirms that the complete chain works: register a user, add at least one emergency contact, log in, press SOS and confirm that the contact receives the alert and the live location link, then mark the user as safe and confirm that tracking stops.

9.8 Localhost Configuration

During development the backend usually runs on the developer's computer, which is referred to as localhost. A phone cannot reach the word "localhost" on a computer, because on the phone it means the phone itself. The mobile application must therefore be given the actual address of the computer on the local network, and the computer and the phone must be on the same network. The computer's firewall must allow incoming connections on the port of the backend. If an emulator is used, the emulator provides a special address through which it can reach the

computer that runs it. Testing the alert chain on localhost proves the logic, but the live location link can be opened by a contact only if the server is reachable from the contact's device.

9.9 Deployment Considerations

If the application is to be used outside the development environment, several points need to be considered.

- **Reachability:** the backend must be reachable from the Internet at a fixed address so that alerts and live location links work from any network.
- **Encryption:** the connection must use encryption, since location and contact data are sensitive.
- **Availability:** the server must be running at all times, because an emergency can happen at any hour.
- **Scalability:** many users may raise alerts at the same time, and the server and database must be able to handle this load.
- **Notification services:** push and SMS services need valid credentials, and SMS may involve cost.
- **Data retention:** live location history should be kept only for a limited time, in line with the privacy-conscious design.
- **Permissions:** the app must request location permission and, for tracking while the screen is locked, must be allowed to run in the background.

9.10 Maintenance and Monitoring

A safety application must be dependable after deployment as well. Some regular activities are needed to keep it so.

- **Monitoring:** the backend should be checked regularly to confirm that it is running and that alerts are being accepted and delivered.
- **Review of delivery logs:** the `delivery_logs` table shows how many alerts were delivered by push and how many needed the SMS fallback, and failures should be investigated.
- **Data clean-up:** location history that has passed its retention period should be deleted.
- **Backups:** the database should be backed up regularly so that user and contact data are not lost.
- **Updates:** the application and the server should be updated when the mobile platform, the mapping API or the notification service changes.
- **User support:** users should be reminded to keep their contacts and permissions up to date.

CHAPTER – 10

CHALLENGES AND LIMITATIONS

10.1 Challenges Faced

The development of an application whose purpose is to work in emergencies brought challenges that are different from those of an ordinary application. The challenges were not only technical. They also concerned the behaviour of a user who is under stress, and the privacy of very sensitive information. The five principal challenges identified during the project are listed in Table 10.1 and are discussed in the sections that follow.

Table 10.1: Challenges and solutions

No.	Challenge	Solution
1	Unreliable networks	The alert pipeline was designed to prioritise the initial SOS message over continuous tracking data when bandwidth is limited.
2	Battery drain from continuous GPS	Polling intervals were tuned and background tracking was optimised to balance accuracy with power usage.
3	Minimising interaction under stress	The interface was reduced to a single prominent button with no confirmation steps.
4	Ensuring guardians actually receive alerts	Delivery status tracking and an SMS fallback were added alongside push notifications.
5	Protecting sensitive location data	Encrypted transmission and secure, time-limited storage of live location history were implemented.

10.2 Location Permission Issues

The application cannot work without the device location. Modern mobile systems require the user to grant location permission explicitly, and the user can withdraw it at any time, or switch off location services altogether. If this happens before an emergency, the SOS button would have no coordinates to send. It is also a challenge to keep tracking active when the screen is locked, since mobile systems restrict background activity to save battery. The application therefore requests the permission at the first suitable moment, before it is needed, tells the user

clearly when location is not available, and is designed so that tracking continues in the background.

10.3 Network Connectivity Issues

Network quality is the most important external factor. A person may be in an area with weak signal, with a mobile data connection that drops and returns, or on a Wi-Fi network without an Internet connection. Any of these can delay or prevent the alert. The design response is to treat the initial SOS alert as the highest-priority message, to keep its payload small, and to hold back the heavier stream of continuous location data until the connection is stable. Testing on strong Wi-Fi, moderate 4G, and weak or intermittent data confirmed that the contacts were notified first and that streaming resumed afterwards.

10.4 Emergency Alert Reliability

The value of the application depends on the guardians actually receiving the alert. A push notification can fail to arrive if the guardian's phone is offline, if notifications are disabled, or if the application of the guardian is not installed. This challenge was addressed in two ways. SMS was added as a fallback to push delivery, and the delivery status of every contact is recorded and returned to the user's device, so that it is possible to know whether the alert reached the guardians. In addition, the throttling and de-duplication of location updates prevent guardians from being overwhelmed with notifications and ignoring them.

10.5 Database Connectivity

The backend depends on the database for every operation: authentication needs user records, the alert needs the contact list, and tracking needs to store coordinates. If the connection between the backend and the database fails, these operations fail. The risk was reduced by

checking the connection settings during setup, by using a limited database user, and by returning clear error responses so that the mobile application can inform the user instead of failing silently. For a larger deployment, a more robust arrangement for database availability would be needed.

10.6 Frontend–Backend Integration

Because the mobile application and the backend are developed as separate parts, they must agree exactly on the format of requests and responses. A mismatch in field names, data formats or addresses stops the two from communicating. During development, the backend requests were first tested independently with an API testing tool, and only then connected to the mobile screens. This approach made it easier to see whether a fault was in the backend or in the application. Using a phone that must reach a backend on a development computer also needs correct network addresses, as discussed in Section 9.8.

10.7 Solutions Applied

To sum up, the solutions applied in the project were: a single-button interface that needs one tap; priority handling of the initial alert; tuned polling intervals with background tracking; SMS fallback with delivery status tracking; throttling and de-duplication of updates; encrypted transmission; secure and time-limited storage of location history; and the practice of testing every module and every request separately before integration.

10.8 Current Limitations

The project has the following limitations, and the team acknowledges them clearly.

- The application depends on the GPS and on a working data connection. Indoors, or in areas with no signal, location accuracy and alert delivery can be reduced.

- If the device has no connectivity at all, the alert cannot be sent. Offline alert queuing is planned as future work.
- The alert reaches only the contacts the user has stored. It does not automatically contact the police or any other official service.
- Activation is by the on-screen SOS button only; the user must be able to reach and unlock the phone. Voice-activated and wearable triggers are future enhancements.
- Continuous GPS tracking uses battery and data, even though polling intervals were tuned to limit this.
- The testing was done with mid-range Android smartphones and simulated scenarios. Wider testing with more devices and in real-world conditions would give stronger evidence.
- The performance observations are qualitative and were not recorded with numerical measurements in the project review.

10.9 Risk Analysis

The main risks to the successful working of the system, together with the measures taken, are summarised in Table 10.2.

Table 10.2: Risk analysis

Risk	Level	Mitigation
No connectivity at the time of SOS	High	Prioritise the initial alert; offline queuing as future work
Location permission withdrawn or GPS switched off	High	Request permission early; inform the user when location is unavailable
Push notification not received	Medium	SMS fallback and delivery status tracking

Risk	Level	Mitigation
Battery drain during tracking	Medium	Tuned polling intervals and optimised background tracking
Misuse or leakage of location data	High	Encrypted transmission, secure storage, time-limited retention
Accidental SOS press	Low	Tracking can be stopped by marking safe or cancelling; contacts can be informed
Backend or database failure	Medium	Clear error responses; monitoring and backups in deployment

CHAPTER – 11

FUTURE ENHANCEMENTS

The present version of the Women Safety App achieves its main goal of providing an instant SOS and continuous live tracking. The project review identified several directions in which the application can be made more capable. These are proposals for future work. None of them is part of the present implementation.

Table 11.1: Future enhancements

Enhancement	Description	Benefit
Police integration	Direct integration with local police control rooms for automated escalation	Official response without depending on contacts
Voice-activated SOS	Hands-free triggering by voice	Useful when the phone cannot be reached
Offline alert queue	Alerts held on the device and sent automatically when connectivity returns	Alerts not lost in dead zones
Wearable integration	SOS trigger from a smartwatch	Faster access to the trigger
Machine learning	Anomaly detection to flag unusual movement patterns	Automatic detection of possible danger
Additional emergency services	Alerts to medical or other services as well as personal contacts	Wider range of help
Improved notifications	Richer and smarter alerts to guardians	Faster and clearer response

11.1 Police Integration

At present, the alert is sent to personal contacts, who must decide what action to take. A valuable extension would be a direct link to the control rooms of the local police, so that an SOS could be escalated automatically with the live location. This would reduce the dependence on the guardians being available and attentive. It would need cooperation with the authorities, agreement on data formats and strict control over misuse, such as false alarms, and over the privacy of the information shared.

11.2 Voice-Activated SOS

In some situations the user cannot reach the phone, or cannot safely look at the screen. A voice-activated trigger, in which a chosen phrase starts the SOS, would allow hands-free activation. The design would need to avoid accidental triggers and to protect the user's privacy while the device listens for the phrase.

11.3 Offline Alert Queue

When there is no connectivity, the alert cannot leave the phone. With offline queuing, the alert and the coordinates would be stored on the device and sent automatically as soon as the connection returns. This extends the idea already implemented, namely prioritising the initial alert under weak connectivity, to the case of complete loss of connection. It would be especially useful in areas with poor coverage, which the project identified as a common setting for real emergencies.

11.4 Wearable Integration

A smartwatch or similar wearable device could offer an even faster way to trigger the SOS, because it is usually within immediate reach. The wearable would send the trigger to the phone, which would then perform the same capture, alert and tracking steps as it does now.

11.5 Machine Learning

Machine-learning-based anomaly detection could study the pattern of movement of the user, such as a sudden change in speed, an unexpected stop or a departure from a usual route, and flag unusual movement automatically. Such a system could prompt the user, or alert the guardians, even if the SOS button has not been pressed. It would need careful design to limit false alarms, and would raise privacy questions that must be handled carefully.

11.6 Additional Emergency Services

Apart from personal contacts and the police, other kinds of help may be needed in an emergency, for example medical assistance. The system could be extended to notify additional services selected by the user, in the same way that contacts are notified at present.

11.7 Improved Notifications

Notifications to guardians could be made more informative, for example by giving a clear summary of the situation and the latest position, and by allowing the guardian to respond within the notification. Better control over the frequency of updates would further improve the balance between timeliness and avoiding alert fatigue. The way the guardian is informed is just as important as the speed of the alert.

11.8 Suggested Order of Implementation

Not all enhancements are equally easy or equally important. Based on the limitations identified in Section 10.8, the following order is suggested. It is a recommendation of the team and not a fixed plan.

Table 11.2: Suggested order for the future enhancements

Order	Enhancement	Reason
1	Offline alert queue	It addresses the most serious limitation, that no alert can be sent without any connection, and it builds on the existing priority-alert design.
2	Improved notifications	It improves the way guardians receive and understand alerts, with a small change to the existing notification module.
3	Voice-activated SOS	It removes the need to reach and unlock the phone, which is a major limitation of the on-screen button.
4	Wearable integration	It adds a faster trigger but needs a new kind of device and a communication link with the phone.
5	Additional emergency services	It widens the help available and depends on agreements with outside services.
6	Police integration	It gives the greatest benefit but needs formal cooperation, strong safeguards and approval from the authorities.

Order	Enhancement	Reason
7	Machine learning	It needs enough movement data and careful handling of false alarms and privacy, so it is the longest-term option.

CHAPTER – 12

CONCLUSION

12.1 Project Summary

The Women Safety App – SOS and Live Tracking is a mobile application that improves personal safety through an instant SOS alert and continuous live location tracking. With a single tap, the application captures the real-time GPS coordinates of the user and sends an emergency alert to the contacts she has chosen. It then keeps sharing her changing location with them until she marks herself safe or cancels the alert. The project was motivated by the fact that traditional methods, such as dialling a number and explaining the situation, are slow and unreliable under threat, and that existing safety applications usually share a location only once.

The system is built as a layered client–server architecture. A mobile application is the frontend, the device GPS and mapping APIs supply location, a backend server processes authentication, alerts and location updates, a database stores users, contacts and events, and a notification service delivers alerts by push and SMS. The work was organised into the SOS Trigger Module, the Live Tracking Module and the Contact and Notification Module, together with the account and profile functions.

12.2 Achievements

Table 12.1: Objectives and their achievement

Objective	Achievement
Single-tap SOS	A single prominent SOS button with no confirmation step alerts the trusted contacts
Continuous live tracking	Location is streamed at regular intervals so that guardians can follow the user until the situation is resolved
Minimal-interaction interface	Dashboard with one large SOS button and three simple cards

Objective	Achievement
Secure contact management	Contacts can be added, edited and removed and are stored on the backend
Dependable delivery under weak connectivity	The initial alert is prioritised over tracking data; SMS fallback and delivery status tracking were added
Data protection	Encrypted transmission and secure, time-limited storage of live location history

In testing, the SOS trigger captured the location and dispatched the alert within a couple of seconds under normal network conditions. Contacts received the alert and a working live-location link, and their view was updated at short regular intervals. Under weak connectivity, the initial alert was delivered before location streaming resumed.

12.3 Benefits

- **For the user:** a faster way to ask for help that requires no explanation, and a sense of security when travelling alone.
- **For families and guardians:** immediate awareness of an emergency and the ability to follow the user's movement instead of guessing.
- **For responders:** a precise and continuously updated location, which helps them reach the user quickly.
- **For society:** a practical example of how technology that people already own can be applied to a social problem.
- **For the team:** experience of building a complete system from requirements to documentation.

12.4 Skills Learned

The project gave the team practical experience in several areas. In mobile application development we learned how to design simple interfaces for use under stress and how to work with the location services of a device. In backend development we learned how to design the services for authentication, alert processing and location updates. In database design we learned to create related tables and to model a real situation with keys and relationships. We learned how push notifications and SMS can be combined for dependable delivery. We also learned about privacy and security when handling location data.

In addition, we learned the practice of the software development process: analysing requirements, drawing architecture and design diagrams, developing modules independently and integrating them, testing under different conditions and measuring performance. Working as a team of three taught us to divide work, to agree on interfaces between modules, and to write clear technical documentation. We are grateful to our guide for the direction she gave us throughout the work.

12.5 Final Conclusion

The Women Safety App shows that a simple, purpose-built application can shorten the gap between the moment a woman feels threatened and the moment help starts to move. By combining a one-tap trigger, continuous live tracking and connectivity-aware alert delivery in one application, it addresses a gap that was found in the existing systems. The architecture is light on the user's device while supporting reliable alerting and tracking through the backend.

The project has limitations, and it does not replace official emergency services. The future enhancements described in Chapter 11, in particular direct police integration, voice-activated

SOS and offline alert queuing, would further strengthen its reliability and reach. We hope that this work will be a useful step towards technology that supports the safety of women, and that it can be developed further.

CHAPTER – 13

REFERENCES

Reports and Research Papers

1. National Crime Records Bureau (NCRB), “Crime in India”, annual report, Ministry of Home Affairs, Government of India.
2. S. Sharma et al., “A Survey on Women Safety Mobile Applications,” International Journal of Computer Applications.
3. R. Kumar and P. Singh, “Real-Time GPS Tracking Systems: A Review,” International Conference on Computing and Communication Technologies.

Documentation and Online Resources

1. Google Developers, “Location and Context – Android Developer Documentation,” developer.android.com.
2. Firebase Documentation, “Firebase Cloud Messaging,” firebase.google.com/docs/cloud-messaging.
3. OpenStreetMap / Google Maps Platform Documentation, “Maps and Location APIs.”

Books

1. Pressman, R. S., and Maxim, B. R., Software Engineering: A Practitioner’s Approach, 8th ed., McGraw-Hill.
2. Sommerville, I., Software Engineering, 10th ed., Pearson.

CHAPTER – 14

APPENDICES

14.1 Application Screenshots

This appendix gives the screenshots of the application. The screenshots of the welcome page, login page, registration page and dashboard were taken from the working application and are reproduced from the project review. Screens for which no screenshot was available are marked for insertion.



Figure 14.1: Welcome page

Welcome Back

Login to Women Safety App

Email / Phone Number

Password

Login

[← Back](#)

Figure 14.2: Login page

Create Account

Register for Women Safety App

Name

Mobile Number

Email

Password

Register

Figure 14.3: Registration page

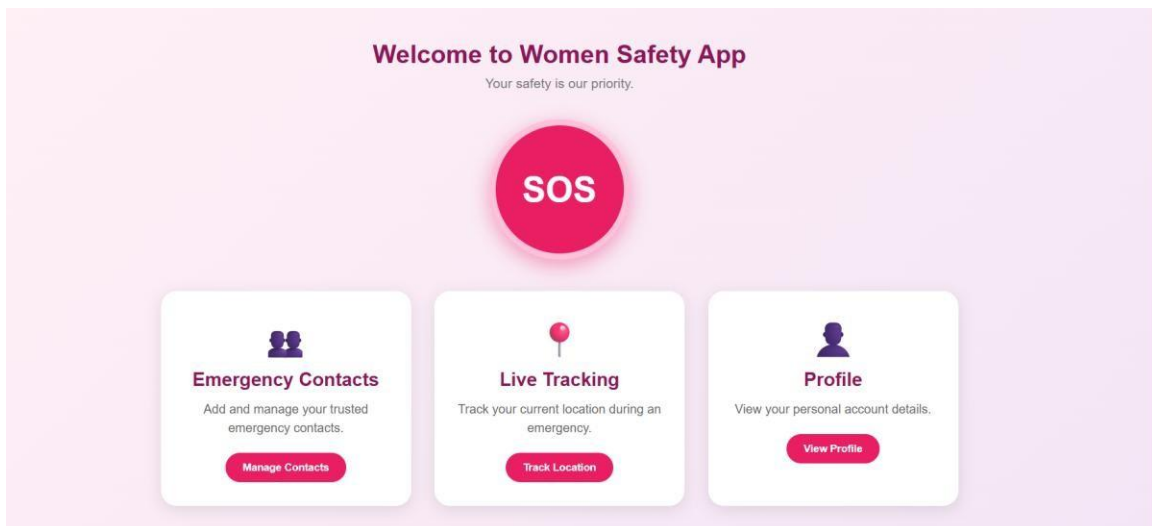


Figure 14.4: Dashboard with SOS button and feature cards

Emergency Contacts

Add people who should be contacted during an emergency.

Add Contact

Ananya 9087654321	Delete
harshika 6985478965	Delete
Udaya 8596745896	Delete

Figure 14.5: Emergency Contacts Page

14.2 Sample Code

The actual source code written for the project is to be placed in this section. The pseudocode below summarises the core logic of the SOS trigger and the live tracking, and matches Chapter 6.

```
// SOS trigger (summary)
ON SOS button press:
    position = getCurrentLocation()
    request  = { userId, lat, lng, time }
    sendWithPriority(request)
    startLiveTracking(userId)

// Live tracking (summary)
WHILE alert is active:
    position = getCurrentLocation()
    IF connection is usable: sendLocationUpdate(userId, position)
    WAIT for polling interval

// Notification (summary)
FOR EACH contact OF user:
    result = sendPush(contact, liveLink)
    IF NOT delivered: result = sendSMS(contact, liveLink)
    saveDeliveryLog(contact, result)
```

14.3 Installation Instructions

The steps below describe the installation of the system in general form. Replace the generic wording by the exact commands and tool names of the project.

1. Install the database software and create the database and tables described in Section 5.4.
2. Install the backend runtime, copy the backend project and install its dependencies.
3. Enter the database, notification and location API settings from Table 9.1 in the backend configuration.
4. Start the backend server and check it with a test request.
5. Open the mobile application project in the development environment and set the backend address.
6. Connect a GPS-enabled Android smartphone, or start an emulator.
7. Build and install the application, and allow the location permission when requested.
8. Register a user, add an emergency contact and test the SOS with a test contact.

14.4 User Manual

14.4.1 Getting Started

1. Install the Women Safety App on your smartphone and open it.
2. On the welcome page, tap **Get Started**.
3. If you are new, tap **Register** and enter your Name, Mobile Number, Email and Password; then press **Register**.
4. If you already have an account, enter your Email or Phone Number and Password and press **Login**.

14.4.2 Add Your Emergency Contacts

Do this before you need the application. From the dashboard, tap **Manage Contacts** under Emergency Contacts. Add the name and phone number of each person you trust, such as parents, siblings or friends. You can edit or remove a contact whenever you wish. Add more than one contact, so that somebody is likely to see the alert quickly.

14.4.3 In an Emergency

Press the large **SOS** button on the dashboard once. You do not need to confirm, type or speak. The app captures your location, sends the alert to all your contacts and begins to share your live location. You may lock the screen or leave the app; sharing continues in the background.

14.4.4 Live Tracking

Your contacts receive a message with a live location link. By opening it they can follow your movement on a map. You can also tap **Track Location** on the dashboard to see your own location during an emergency.

14.4.5 When You Are Safe

When you are safe, mark yourself as safe, or cancel the alert. Location sharing stops immediately. If you pressed SOS by mistake, cancel it and inform your contacts.

14.4.6 Profile and Logout

Tap **View Profile** to see your account details. Log out when you have finished, especially if other people use your phone.

14.4.7 Good Practices

- Keep the location (GPS) switched on and grant the location permission to the app.
- Keep your phone charged. Continuous tracking uses battery.
- Keep your contacts up to date and tell them that you have added them.
- Test the app once with a trusted contact, so that both of you know what the alert looks like.
- Remember that the app supports, but does not replace, the police and other official emergency service.

14.5 Abbreviations and Glossary

Table 14.1: Abbreviations and terms used in this report

Term	Meaning
SOS	A distress signal; in this application, the one-tap emergency alert
GPS	Global Positioning System; satellite-based positioning used to obtain latitude and longitude
API	Application Programming Interface; a defined way for software components to communicate
Push notification	A message delivered instantly to the screen of a smartphone by a notification service
SMS	Short Message Service; text message used as a fallback for alert delivery
Guardian / emergency contact	A trusted person chosen by the user to receive her SOS alerts
Live location link	A link sent to a contact through which the user's current position can be followed
Polling interval	The time between two successive readings of the GPS position
Throttling	Limiting how often updates are sent
De-duplication	Dropping repeated identical updates
Backend	The server-side part of the system
Frontend	The mobile application part of the system
ER diagram	Entity-relationship diagram showing database entities and their relations
DFD	Data flow diagram showing the movement of data through the system
Term	Meaning
PK / FK	Primary key / foreign key in a database table
CSIT	Computer Science and Information Technology

14.6 Requirements Traceability Matrix

The matrix below links each functional requirement of Section 2.4 to the module that implements it, the section of the report where it is described and the test case that checks it.

Table 14.2: Requirements traceability matrix

ID	Requirement	Module	Section	Test case
FR1	User registration	User Account	6.2	TC01–TC03
FR2	User login	User Account	6.2	TC04–TC06

FR3	Dashboard	Frontend	6.12	TC04
FR4	Contact management	Contact and Notification	6.4	TC08–TC12
FR5	One-tap SOS	SOS Trigger	6.5	TC13
FR6	Location capture	SOS Trigger	6.6	TC13, TC22
FR7	Alert delivery	Contact and Notification	6.8	TC14, TC17
FR8	Fallback delivery	Contact and Notification	6.8	TC18
FR9	Live tracking	Live Tracking	6.7	TC15, TC21
FR10	Delivery status	Contact and Notification	6.8	TC18
FR11	Stop SOS	Live Tracking / Backend	6.7, 7.10	TC19, TC20
FR12	Profile	Profile and Secure Storage	6.3	—
FR13	Logout	User Account	7.12	TC07